arXiv:2605.15422v4 [cs.LG] 7 Aug 2026

DualKV: Shared-Prompt Flash Attention for Efficient RL Training with Large Rollouts and Long Contexts

Jiading Gai[1,*] **Shuai Zhang**[1,*] **Xiang Song**[2†] **Bernie Wang**[1] **George Karypis**[3†]

[1]*Amazon Web Services* [2]*Google* [3]*University of Minnesota*

{jiadingg, shuaizs, yuyawang}@amazon.com xiangsx@google.com karypis@umn.edu

[*]*Equal contribution.*

https://github.com/amazon-science/dualkv-flash-attn-for-rl

Abstract

Modern RL post-training methods such as GRPO and DAPO train on N response sequences of R tokens sampled from a shared prompt of P tokens, but standard FlashAttention replicates all P prompt tokens N times across both forward and backward passes — duplicating compute and memory on identical hidden states. In large-rollout, long-context RL training ($N \geq 16$, $P \geq 8$K), this redundancy dominates the policy update cost. We observe that in decoder-only models, causal masking makes prompt representations invariant across sequences at every layer, so all per-token operations (norms, projections, MLP) and attention can process the prompt once — a property not yet exploited at the kernel level for training. We propose **DualKV**, the first FlashAttention kernel variant that eliminates shared-prompt replication during RL training, via (1) fused CUDA forward and backward kernels that iterate over two disjoint KV regions — shared context and per-sequence response — in a single kernel launch, and (2) a data-pipeline redesign in veRL that repacks $N(P+R)$ tokens into $P+NR$ tokens per micro-batch, extending the token reduction from attention to the entire model by a factor $\rho = N(P+R)/(P+NR)$. DualKV is mathematically equivalent to standard attention and introduces no approximation. On Qwen3-8B GRPO training with 8×H100 GPUs (N=32, 8K-context), DualKV achieves 1.63–2.09× policy-update speedup, enables 2× larger micro-batches, and raises MFU from 36% to 76%. Similar gains hold for DAPO (2.47× speedup, 77% MFU). At 30B MoE scale on 16×H100, DualKV achieves 3.82× policy-update and 3.38× end-to-end step speedup over FlashAttention (which requires 4-way Ulysses sequence parallelism to avoid OOM). DualKV also extends to hybrid sliding/global attention with head dimension 512 (which FA2 does not support) and integrates with Ulysses sequence parallelism, demonstrated on Gemma-4-31B GRPO at 64K context.

1 Introduction

Modern RL post-training methods such as GRPO [Shao et al., 2024] and DAPO [Yu et al., 2025] compute log-probabilities and gradients for N response sequences that share a prompt of P tokens. With standard FlashAttention-2 (FA2) [Dao, 2024], the training micro-batch contains N sequences of length $S_i = P + R_i$, repeating the P-token prompt N times. Each forward and backward pass recomputes K, V activations and their gradients over $(N-1) \times P$ redundant prompt tokens per layer.

Recent work demonstrates that model accuracy continues to improve with increasingly large rollout factors. Lightman et al. [2024] show that best-of-N performance on the MATH dataset scales log-

[†]Work done while at AWS.

linearly up to $N{=}1860$. GRPO and related methods similarly benefit from large N for more accurate reward signal estimation (e.g., DAPO uses $N{=}16$). However, the compute cost of the standard N-copy packing grows as $O(N \cdot S^2)$: each of the N sequences independently recomputes attention over the full P prompt tokens, duplicating $(N{-}1) \times P$ prompt tokens in both compute and memory at every layer. As we show in Section 4.2, eliminating this replication cuts policy-update time by $2\times$.

Prompt KV sharing for inference has been addressed from multiple angles: paged attention [Kwon et al., 2023] and prefix caching avoid redundant prompt KV storage via copy-on-write block tables, while bifurcated attention [Athiwaratkun et al., 2024] decomposes attention into shared-prompt and per-sequence phases for parallel decoding. However, these mechanisms target inference only, where shared prompt KV is read-only; they have no training backward, no autograd-compatible gradient accumulation for the shared prompt, and cannot be dropped into a RL policy-update pipeline (Appendix J). Training raises three new challenges: (1) the backward pass has N sequences concurrently accumulating gradients into the shared KV buffer — a pattern absent from inference that requires both atomic accumulation for race-free writes and fp32 accumulators with a final cast to match FA2's per-element precision; (2) autograd must aggregate gradients correctly across the context self-attention and decoded-attention calls that both touch the shared KV; and (3) the RL training framework's data pipeline must be restructured to group same-prompt responses per micro-batch (Appendix B). DualKV addresses all three at the kernel and system level.

We observe that in decoder-only models, prompt hidden states are *identical* across all N sequences at every layer: causal masking ensures each prompt token attends only to preceding prompt tokens, never to the differing response tokens that follow, so the prompt representations are independent of which response is generated. Prefix Grouper [Liu et al., 2025] applies this at the framework level but provides no kernel support — it still passes the full N-replicated KV through standard FA2, retaining the $O(N \cdot P \cdot d)$ memory bottleneck that causes OOM and forces sequence parallelism. DualKV contributes the first FlashAttention kernel variant that eliminates this replication, with fused forward and backward CUDA kernels that read shared KV from a single physical buffer and accumulate gradients from N concurrent sequences via fp32 atomic writes. By packing the micro-batch with a single prompt copy and decomposing attention into context self-attention (computed once) and a fused DualKV kernel for decoded attention, DualKV eliminates redundant prompt computation throughout the entire model — not just attention, but also norms, projections, and MLP layers — yielding both memory and compute savings without approximation. DualKV also supports head dimension 512 and hybrid sliding/global attention (Gemma-4, beyond FA2's $d{\leq}256$ limit) and composes with Ulysses sequence parallelism for ultra-long context; we validate both with end-to-end GRPO on Gemma-4-31B at up to 64K, extending DualKV to the hybrid sliding/global attention adopted by recent model architectures.

2 Eliminating Prompt Replication in RL Training: The DualKV Design

Each GRPO training step processes N response sequences of length $S_i = P + R_i$ that share a prompt of P tokens. The step runs three passes before the optimizer step: (1) **old_log_prob**: forward through $\pi_{\theta_{\text{old}}}$ for the importance ratio; (2) **ref_log_prob**: forward through π_{ref} for the KL penalty; and (3) **policy update**: forward+backward through π_θ producing the policy gradient. All three passes see the same packed micro-batch with $(N{-}1) \cdot P$ redundant prompt tokens, so DualKV accelerates all three. The same applies to any RL method that packs N same-prompt responses, e.g., DAPO.

2.1 Baseline: Standard Packing with Replicated Prompts

Standard implementations (e.g., veRL with FA2) pack the N sequences into a single `flash_attn_varlen_func` call using cumulative sequence lengths:

$$\underbrace{[P, R_1]}_{\text{seq 1}},\ \underbrace{[P, R_2]}_{\text{seq 2}},\ \ldots,\ \underbrace{[P, R_N]}_{\text{seq } N} \quad \text{— total } T_{\text{std}} = N(P+R) \text{ tokens} \tag{1}$$

The prompt tokens are *replicated* N times: each sequence carries its own copy of the prompt's hidden states, QKV projections, and attention computation. Every per-token operation (Norm, QKV projection, RoPE, MLP, output projection) processes all $N(P + R)$ tokens. For attention, FA2

computes N independent causal self-attention passes, one per sequence i:

$$O^{(i)} \;=\; \text{softmax}\left(\frac{Q^{(i)}(K^{(i)})^\top}{\sqrt{d}} + M_{\text{causal}}\right) V^{(i)}, \qquad i = 1, \ldots, N, \tag{2}$$

where $Q^{(i)}, K^{(i)}, V^{(i)} \in \mathbb{R}^{(P+R_i)\times H\times d}$ are sequence i's packed projections and M_{causal} is the per-sequence causal mask. FA2 computes Eq. 2 via tile-wise online softmax. FLOPs: $O(N \cdot S_i^2 \cdot H \cdot d)$.

2.2 DualKV: Single-Prompt Packing

Prompt invariance and redundancy. In Eq. 2, split the output rows of $O^{(i)}$ into prompt-query rows $O_P^{(i)}$ (the first P) and response-query rows $O_{R_i}^{(i)}$ (the remaining R_i). For the prompt-query rows, causal masking blocks attention to response keys, so only the prompt keys participate:

$$O_P \;=\; \text{softmax}\left(\frac{Q_P(K_P)^\top}{\sqrt{d}} + M_{\text{causal},P}\right) V_P, \tag{3}$$

where Q_P, K_P, V_P are the prompt-row projections. Because all N sequences share the same prompt tokens, Q_P, K_P, V_P are identical across i (Appendix A.2), so O_P is the same for every sequence. The $N-1$ prompt-on-prompt computations in the baseline are therefore redundant.

For the response-query rows, each response attends causally to *both* the shared prompt keys and its own response keys:

$$O_{R_i}^{(i)} \;=\; \text{softmax}\left(\frac{Q_{R_i}^{(i)}[K_P;\, K_{R_i}^{(i)}]^\top}{\sqrt{d}} + M_{\text{causal}}\right) [V_P;\, V_{R_i}^{(i)}], \tag{4}$$

where $[\cdot\,;\,\cdot]$ denotes concatenation along the token axis. This block is not redundant — each response has distinct queries — but it reuses the same K_P, V_P for all i.

DualKV packing. DualKV packs the micro-batch as a single shared prompt followed by N per-sequence responses:

$$\underbrace{[P]}_{\text{prompt (once)}}, \underbrace{[R_1]}_{\text{resp 1}}, \ldots, \underbrace{[R_N]}_{\text{resp } N} \quad \text{— total } T_{\text{dk}} = P + NR \text{ tokens} \tag{5}$$

All per-token operations (norms, projections, RoPE, MLP, output projection) now process $P + NR$ tokens instead of $N(P+R)$ — saving $(N-1)P$ tokens throughout the entire model.

Two-call decomposition. Eq. 3 and Eq. 4 can be computed separately to eliminate the redundancy while preserving exact attention:

- **Call 1 — context self-attention** (`flash_attn_varlen_func`, standard FA2): computes Eq. 3 over a single copy of the prompt, *once*. FLOPs: $O(P^2 \cdot H \cdot d)$.
- **Call 2 — decoded attention** (`flash_attn_dualkv_varlen_func`, DualKV kernel): computes Eq. 4 for all N responses, attending to the shared K_P, V_P (from Call 1) plus per-sequence K_{R_i}, V_{R_i}. FLOPs: $O(N \cdot R \cdot S \cdot H \cdot d)$.

Why a new kernel. Eq. 4's attention pattern — response queries reading from the concatenation $[K_P; K_{R_i}]$ — cannot be consumed by standard FA2 without materializing N copies of K_P, V_P (reintroducing the replication we just eliminated). The DualKV kernel iterates over the two physically disjoint KV regions (K_P, V_P shared across the batch; K_{R_i}, V_{R_i} per-sequence) within a single launch. The kernel's interface and algorithms are detailed in Section 3.

Data pipeline. This packing requires that all N responses from the same prompt are co-located on the same GPU within the same micro-batch. veRL's rollout engine already produces prompt-contiguous sequences; DualKV preserves this ordering by skipping veRL's `balance_batch` step (which reorders by sequence length) and disabling intra-epoch shuffle in the mini-batch iterator. The resulting pipeline preserves the exact mini-batch gradient estimator (Appendix B). When the micro-batch contains multiple prompt groups, DualKV handles each group independently with its own context KV and per-group `cu_seqlens`, supporting heterogeneous prompt lengths within a single forward pass.

Gradient correctness. The shared K_P, V_P receive gradients from both calls — autograd sums the two contributions, while the DualKV kernel internally accumulates the N per-sequence terms within Call 2:

$$\frac{\partial \mathcal{L}}{\partial K_P} \;=\; \left(\frac{\partial \mathcal{L}}{\partial K_P}\right)_{\text{Call 1}} \;+\; \sum_{i=1}^{N} \left(\frac{\partial \mathcal{L}}{\partial K_P}\right)^{(i)}_{\text{Call 2}} \tag{6}$$

(same structure for V_P). The first term is Call 1's prompt self-attention gradient; the $\sum_{i=1}^{N}$ collects Call 2's per-response contributions, creating an N-way concurrent write into shared K_P, V_P — a pattern with no analog in FA2's backward, resolved at the kernel level in Section 3.2. The total is mathematically identical to the FA2 baseline (Appendix A.5).

When DualKV helps. The token reduction ratio $\rho = N(P+R)/(P+NR)$ governs the per-token operation speedup (norms, projections, RoPE, MLP, output projection) throughout the entire model; attention has additional structural savings (Appendix C). ρ is largest in two regimes: **large rollout** ($N \geq 16$, common in GRPO and DAPO; at N=32, P=16K the reduction reaches 7.2×) and **long prompts** ($P \geq$ 16K, prevalent in agentic tasks [Jimenez et al., 2024] and repository-level code generation [Liu et al., 2024b]; at P=64K, N=16 the reduction reaches 14.3×). In practice, ρ is bounded by the per-GPU micro-batch size (Section 4.2). See Appendix C.3 for the full analysis.

2.3 Composing DualKV with Sequence Parallelism

DualKV and sequence parallelism (SP) are two orthogonal techniques for scaling long-context training: DualKV deduplicates the shared prompt *across* the N sequences of a group, while Ulysses SP shards *one* packed sequence across ranks. Scaling long-context training commonly requires SP, which adds step latency through per-layer all-to-all communication. In GRPO/DAPO, where N responses share a large P-token prompt, the dominant redundancy is prompt replication: SP shards this replicated $N(P+R)$ sequence across ranks but never removes the redundancy, paying all-to-all communication to distribute it, whereas DualKV eliminates the replication at its source ($N(P+R) \to P+NR$) — a more efficient approach for shared-prompt RL. SP remains necessary at ultra-long context, where even the deduplicated $P+NR$ sequence exceeds one rank, so DualKV and SP must be used together. Making them work together requires two adaptations.

Repack before slice; all-to-all inside attention. The DualKV repack (Section 2.2) runs *before* the Ulysses sequence slice, so the shared prompt stays a single contiguous block; the resulting $P+NR$ sequence is then padded to a multiple of the SP degree and sliced contiguously across ranks like any sequence, and all packed side tensors (rolled labels, per-token temperature) are sliced identically so they stay aligned. Attention is restored exactly as in standard Ulysses SP: an all-to-all before the DualKV kernel gathers the full token sequence while scattering heads (each rank holds all $P+NR$ tokens but H/SP heads), the two-region DualKV kernel runs unchanged on that per-rank shard, and a second all-to-all after attention scatters the sequence back and gathers heads. Grouped-query attention is handled by repeating KV heads to a multiple of the SP degree before the head scatter, exactly as in the FA2 SP path. The DualKV repack, kernel, and per-group `cu_seqlens` are unchanged under SP; the only SP-specific operations are the two all-to-alls wrapping attention.

Shared prompt logits under SP. The other place DualKV interacts non-trivially with SP is the fused log-probability head, which — during the training forward pass — computes each already-generated token's log-prob for the policy loss without materializing full logits. DualKV's packing places each prompt group's prompt once, so the first token of *every* response in the group is predicted by the logits at the last prompt position $P-1$; the fused log-prob head computes these first-token log-probs from that shared row (the remaining tokens use the standard shifted log-probs). Under SP the packed sequence is sliced contiguously, so this global position $P-1$ resides on exactly *one* rank, while the responses that need it are spread across *all* ranks. We resolve this with a lightweight collective. Let p_g be the global index of group g's last prompt token in the packed sequence (Eq. 5), and $\text{parts}=\lceil (P+NR)/\text{SP} \rceil$ the per-rank shard length; then rank $\text{owner}(g)=\lfloor p_g/\text{parts} \rfloor$ holds that token's hidden state at local index $\text{local}(g)=p_g \bmod \text{parts}$. Each rank projects only the shared rows it owns and an all-reduce over the SP group reconstructs, for every group, the exact SP=1 shared hidden state h_{p_g}:

$$\tilde{h}_g \;=\; \sum_{r=0}^{\text{SP}-1} \mathbb{1}[\text{owner}(g)=r]\; h^{(r)}_{\text{local}(g)}, \qquad \ell_g \;=\; \tilde{h}_g\, W_{\text{LM}}^{\top}/\tau, \tag{7}$$

where $h^{(r)}$ is rank r's local hidden shard and ℓ_g the shared prompt's logits at the last position (temperature τ) from which each response's first-token log-prob is read. Since each position has exactly one owner, the indicator sum reconstructs h_{p_g} exactly. The payload is G hidden vectors for G prompt groups, so communication is $O(G\,d)$, independent of context length. A parity test confirms the first-token log-probs agree with the SP=1 path to max$|\Delta|$=0 across SP$\in\{1, 2, 4, 8\}$ (Section 4.5).

3 New Computational Primitive: The DualKV Kernel

This section describes the DualKV CUDA kernel that executes Call 2 of the two-call decomposition (Section 2.2). Implemented as an extension of FA2's codebase, the kernel accepts five input tensors — a decoded query q, the shared context K_c, V_c (single copy, stored once), and per-sequence varlen-packed decoded K_d, V_d, plus a `context_seqlen` $= P$ parameter that shifts the causal mask so decoded queries attend to all P context keys and their own preceding decoded keys. We write K_c, V_c and K_d, V_d (following FlashAttention's naming) for K_P, V_P and K_{R_i}, V_{R_i} in Section 2.2. Full Python signature in Appendix H; pseudocode in Appendix G; theoretical analysis in Appendix C.

Head dimensions and sliding-window support for hybrid architectures. Recent open models interleave two attention layer types that fall outside FA2's supported range. Gemma-4-31B, for example, stacks full-attention layers at head dimension 512 with sliding-window layers at head dimension 256 — and FA2 supports only $d \leq 256$, so it cannot serve the d=512 global layers at all. To bring shared-prompt deduplication to such architectures, we add two DualKV kernel variants specifically for Gemma-4 support: a head-dim-512 full-attention path for the global layers, and kernel-native causal sliding-window attention for the d=256 local layers (mathematically equivalent to windowed FA2, verified against it). This lets a single DualKV forward run the full hybrid stack, dispatching per layer by head dimension and window. Implementation details — tile sizes and the shared-memory/register strategy that makes d=512 fit — are in Appendix I.

3.1 Forward Pass

The DualKV forward kernel extends FA2's tiled online-softmax algorithm (tile size B_N) to iterate over two physically disjoint KV regions within a single kernel launch. For each decoded query $q_r^{(i)}$ (position r in sequence i), the kernel:(1) **Context phase**: Iterate over $\lceil P/B_N \rceil$ tiles of $k_{\text{ctx}}, v_{\text{ctx}}$ (tile-level slices of K_c, V_c; lowercase denotes per-tile data following FA2's convention), shared across all N sequences in the grid. Accumulate running softmax statistics $(m, \ell, \mathbf{o})$ using the online-softmax algorithm; (2) **Decoded phase**: Continue iteration over $\lceil R_i/B_N \rceil$ tiles of sequence i's own $k_{\text{dec}}^{(i)}, v_{\text{dec}}^{(i)}$. The running statistics carry over from the context phase, so the two-phase iteration produces the same output as if $[k_{\text{ctx}}; k_{\text{dec}}^{(i)}]$ were stored contiguously; (3) **Causal masking**: All context positions ($j < P$) are unmasked for every decoded query; decoded positions are masked causally using the P-shifted logical position $P + r$ (the `context_seqlen` parameter).

The kernel decouples the physical block index from the logical key position: physical block n has logical start $n \cdot B_N$ when $n < \lceil P/B_N \rceil$ (context) and $P + (n - \lceil P/B_N \rceil) \cdot B_N$ otherwise (decoded), so the causal mask operator sees the two physically disjoint regions as a single contiguous causal sequence.

Per-tile out-of-bounds (OOB) masking. Because the context and decoded regions are tiled to B_N independently, up to two tiles within a single kernel instance can be partial — the last context tile (when $P \bmod B_N \neq 0$) and the sequence's last decoded tile (when $R_i \bmod B_N \neq 0$) — rather than FA2's single-tile boundary. The kernel bounds-checks every tile load and applies the per-tile OOB mask on every iteration, forgoing FA2's fast-path unmasked-steps loop. This cost is negligible compared to the $(N-1) \cdot \lceil P/B_N \rceil$ context-tile passes eliminated.

The kernel supports grouped-query attention (GQA) and variable decoded lengths (via `cu_seqlens_q`). Its output matches Eq. 4 exactly; full pseudocode is in Algorithm 3 (Appendix G), and the full derivation is in Appendix A, §A.3.

3.2 Backward Pass with fp32 Atomic Accumulation

The backward pass computes gradients for all five inputs (dQ, dK_c, dV_c, dK_d, dV_d) via the main DualKV backward kernel (Algorithm 1) and three auxiliary launches (described below). Two

pieces distinguish it from FA2's backward: (1) **Two-region grid iteration** mirroring the forward (Section 3.1): per-block pointer dispatch selects K_c/V_c versus K_d/V_d, logical key positions feed the causal mask, and per-tile OOB masking handles two independent partial tiles. (2) **fp32 atomic accumulation** for the shared context gradients, resolving a concurrent-write race and the precision loss from accumulating N partial gradients in bf16, both unique to DualKV's shared context buffer.

Context gradient structure. The loss gradient with respect to the shared context KV is a sum over all N sequences in the batch:

$$\frac{\partial L}{\partial K_c} = \sum_{i=1}^{N} \left(\frac{\partial L}{\partial K_c}\right)^{(i)}, \quad \frac{\partial L}{\partial V_c} = \sum_{i=1}^{N} \left(\frac{\partial L}{\partial V_c}\right)^{(i)}, \tag{8}$$

where $(\partial L/\partial K_c)^{(i)}$ is the contribution from sequence i's decoded queries attending to the context (closed form in Appendix A, §A.4). This additive structure drives the backward kernel design: the N summands are computed by N separate thread blocks, creating the concurrent-write problem Algorithm 1 resolves.

Algorithm 1 DualKV backward main kernel (launched on grid $(n_{\text{ctx}} + n_{\text{dec}},\ N,\ H)$)

Require: dO, O, Q, saved `softmax_lse`, K_c, V_c, K_d, V_d
Require: fp32 scratch $dK_{c,\text{acc}}, dV_{c,\text{acc}}$ (shared across batch; zero-initialized); dQ_{acc} (per batch, fp32)
Require: $D = \text{rowsum}(dO \odot O)$ (from `dot_do_o`, reused from FA2)
1: **for** each (n, b, h) **in parallel do** ▷ grid: (K-col block, batch, head)
2: $n_{\text{ctx}} \leftarrow \lceil P/B_N \rceil$; `is_context` $\leftarrow (n < n_{\text{ctx}})$
3: **if** `is_context` **then**
4: $(K_n, V_n) \leftarrow (K_c, V_c)[nB_N : (n{+}1)B_N]$; $j_{\text{base}} \leftarrow nB_N$
5: **else**
6: $(K_n, V_n) \leftarrow (K_d^{(b)}, V_d^{(b)})[(n{-}n_{\text{ctx}})B_N : \ldots]$; $j_{\text{base}} \leftarrow P + (n{-}n_{\text{ctx}})B_N$
7: **end if**
8: $acc_{dK}, acc_{dV} \leftarrow 0$ in fp32
9: **for** each Q-row block Q_m in causal support of K_n **do**
10: Load $Q_m, dO_m,$ `softmax_lse`$[m], D_m$ from HBM to on-chip SRAM
11: On chip, compute $P = \exp(Q_m K_n^\top/\sqrt{d} - \texttt{lse}_m)$ with causal mask ▷ per-tile OOB mask if partial
12: acc_{dV} += $P^\top dO_m$
13: $dP \leftarrow dO_m V_n^\top$; $dS \leftarrow P \odot (dP - D_m)$
14: acc_{dK} += $dS^\top Q_m/\sqrt{d}$
15: `atomicAdd`$(dQ_{\text{acc}}^{(b)}[m],\ dS\, K_n/\sqrt{d})$ ▷ FA2's existing dQ pattern
16: **end for**
17: **if** `is_context` **then** ▷ N **blocks (one per** b**) race on same address**
18: `atomicAdd`$(dK_{c,\text{acc}}[nB_N],\ acc_{dK})$
19: `atomicAdd`$(dV_{c,\text{acc}}[nB_N],\ acc_{dV})$
20: **else** ▷ Decoded block: exclusive per-b region, no race
21: $dK_d^{(b)}[\ldots] \leftarrow \text{bf16}(acc_{dK})$; $dV_d^{(b)}[\ldots] \leftarrow \text{bf16}(acc_{dV})$
22: **end if**
23: **end for**

Concurrent-write problem (race on line 18). In a standard FA2 backward, the grid is (n, b, h) and each sequence i holds a private $K_c^{(i)}, V_c^{(i)}$, so the batch index maps each block to a separate $dK_c^{(i)}$ region — each block can cast its fp32 accumulator directly to bf16 and write with no contention. DualKV collapses this batch dimension for the shared context: the single dK_c and dV_c buffer has no batch offset, yet N thread blocks — one per decoded sequence — concurrently reach Algorithm 1's context epilogue. A standard direct-write epilogue would produce a data race. Resolving the race via bf16 `atomicAdd` would compound $N{-}1$ bf16 rounding steps on top of the initial cast, turning the cross-sequence reduction itself into a half-precision accumulation.

fp32 atomic accumulation. We resolve this with a two-stage design. Stage 1 (Algorithm 1): context blocks `atomicAdd` into an fp32 scratch buffer; decoded blocks bypass the scratch and write bf16 directly (they own disjoint dK_d/dV_d regions). Stage 2 (Algorithm 2): after all Stage 1 blocks complete, a separate kernel casts the fp32 scratch to bf16. This reproduces FA2's single-cast precision profile despite the N-way concurrent write: all N context contributions are summed in fp32, and only the final value is cast to bf16.

Algorithm 2 `convert_dkv_context`: fp32-to-bf16 cast, launched on grid $(n_{\text{ctx}},\ 1,\ H)$ after Algorithm 1 completes (batch dimension collapsed because context is shared)

Require: fp32 scratch $dK_{c,\text{acc}}, dV_{c,\text{acc}}$ (aggregated over N sequences by Algorithm 1)
1: **for** each (n, h) **in parallel**, and each (token, dim) in tile **do**
2: $\quad dK_c[nB_N{+}\text{token}, \text{dim}] \leftarrow \text{bf16}(dK_{c,\text{acc}}[nB_N{+}\text{token}, \text{dim}])$
3: $\quad dV_c[nB_N{+}\text{token}, \text{dim}] \leftarrow \text{bf16}(dV_{c,\text{acc}}[nB_N{+}\text{token}, \text{dim}])$
4: **end for**

Analogy to FA2's dQ pipeline. FA2 uses an fp32-then-cast pipeline for dQ under K-block-parallel scheduling: the main backward kernel atomic-adds into dQ_{acc} (Algorithm 1's dQ `atomicAdd` line), and a separate `convert_dq` kernel casts dQ_{acc} to bf16. DualKV replicates this pipeline for dK_c, dV_c — now indexed by *batch element* (not K-block) — with Algorithm 1's context-epilogue atomicAdd and the separate `convert_dkv_context` kernel (Algorithm 2) casting to bf16. The fp32 accumulator keeps the context gradient within standard fp32 rounding of the FA2 baseline; formal equivalence is proven in Section A.5.

4 Experiments

We first benchmark the DualKV attention kernel in isolation (Section 4.1), then evaluate end-to-end DAPO and GRPO training at H100 scale on LongReason (Section 4.2) and on multi-node MoE (Section 4.3). We also study the memory scaling with context length (Section 4.4). Appendix E.1 confirms DualKV benefits extend to the short-prompt regime (GSM8K, A100). Our primary baseline is FA2 in veRL's FSDP worker [Sheng et al., 2025]; we also include FA3 in Section 4.2.

4.1 Kernel-Level Speedup: Forward, Backward, and Combined

Table 1 reports separate forward-only, backward-only, and combined wall-clock times on a single NVIDIA A100 GPU with Qwen3-8B model dimensions (H=32, H_k=8, d=128, GQA 4:1), fp16 with gradient computation enabled. The backward pass has different compute characteristics than the forward — it computes gradients $\partial\mathcal{L}/\partial Q$, $\partial\mathcal{L}/\partial K$, $\partial\mathcal{L}/\partial V$ and involves the gradient accumulation for shared K_c, V_c described in Section 3.2 (Appendix A, Eqs. 22–23). DualKV gradients match FA2 to half-precision rounding (verified via `torch.allclose` with atol=10^{-3}, rtol=10^{-3}).

Table 1: FA2 vs. DualKV wall-clock time (ms) and peak memory across varying rollout factor N and prompt length P; response length R=2048 throughout. OOM = out of memory.

		FA2 (ms)			**DualKV (ms)**			**Speedup**			**Mem**
N	P	fwd	bwd	f+b	fwd	bwd	f+b	fwd	bwd	f+b	↓
28	4K	43.6	128.6	172.2	27.1	77.1	104.2	1.61×	1.67×	1.65×	63%
28	16K	381.3	1059.6	1441.0	98.7	273.0	371.7	3.86×	3.88×	3.88×	85%
16	32K	771.5	2111.8	2883.4	139.5	386.2	525.8	5.53×	5.47×	5.48×	86%
16	64K		OOM		363.3	994.5	1357.8		∞		—

Combined fwd+bwd speedup ranges from 1.65× (P=4K) to 5.48× (P=32K), with up to 86% peak memory reduction; at $P \geq$ 32K standard FA2 OOMs entirely. Backward speedup tracks forward within 3% as expected: the attention backward computes the same $QK^\top$ products as the forward. The backward-to-forward ratio ($\sim$2.7×) is stable across all configurations for both FA2 and DualKV, confirming that DualKV's two-region backward kernel adds no structural overhead beyond attention's intrinsic backward cost. Gradient accumulation for shared K_c, V_c (Section 3.2) adds no measurable overhead — fp32 atomic writes and the follow-up bf16 cast are bandwidth-bound on context positions only and land inside the backward's GEMM timing.

Comparison with Prefix Grouper. Table 2 benchmarks against Prefix Grouper (PG) [Liu et al., 2025], a framework-level shared-prompt optimization for training (no kernel-level deduplication) — FA2 still receives N-replicated KV on padded tensors, so projections, MLP, and norms process the full N-copy prompt. DualKV is 2–4× faster and uses 3–5× less memory than PG across

Table 2: Single-layer fwd+bwd: DualKV vs. PG vs. FA2 (Qwen3-8B, H100). Full sweep in Appendix E.4.

		Time (ms)			**DK Speedup**		**Mem (GB)**	
P	mb	FA2	DualKV	PG	vs FA2	vs PG	DK	PG
5K	32	597	**180**	626	3.32×	3.48×	**13.7**	69.8
8K	16	549	**128**	465	4.28×	3.63×	**9.9**	50.7
16K	8	553	**125**	435	4.41×	3.47×	**9.0**	45.9
32K	4	810	**246**	514	3.29×	2.08×	**12.5**	43.0
65K	4	OOM	**654**	OOM	∞	∞	**22.7**	OOM
131K	8	OOM	**2097**	OOM	∞	∞	**44.5**	OOM

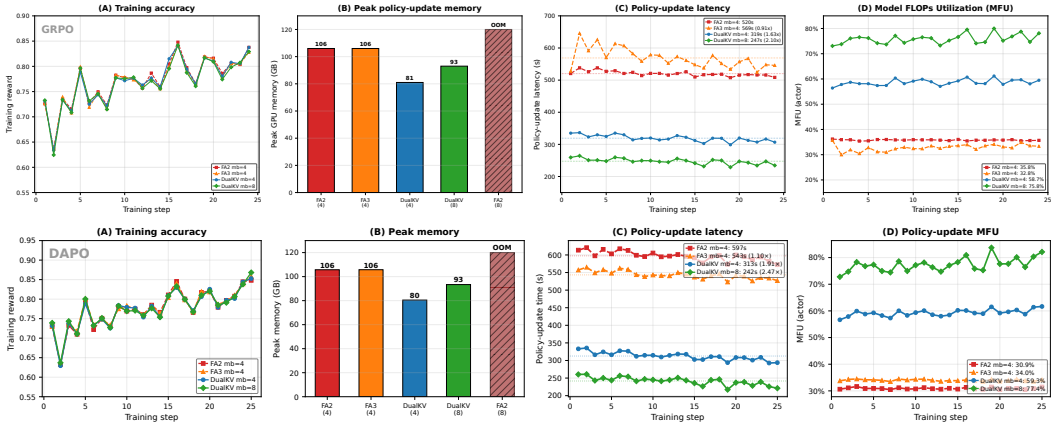

Figure 2: LongReason training (Qwen3-8B, 8×H100), four configurations. **Top row**: GRPO (N=32, steps 1–24). **Bottom row**: DAPO (N=32, steps 1–25). Panels: **(A)** Training reward — all configs track identically. **(B)** Peak memory. **(C)** Policy-update latency. **(D)** Policy-update MFU. GRPO: DualKV mb=8 achieves 2.09× speedup, 75.8% MFU. DAPO: 2.47× speedup, 77.4% MFU.

all configurations where both run. At high micro-batch (P=5K, mb=32), PG's padded representation makes it even slower than FA2 varlen. At $P \geq$32K, both FA2 and PG OOM while DualKV continues to scale. PG only optimizes attention (∼30% of layer FLOPs), while DualKV eliminates redundancy across all operations via packed varlen representation.

4.2 End-to-End GRPO and DAPO Training on LongReason

To validate DualKV in a higher-ρ regime with longer prompts and larger rollout factors, we evaluate on LongReason [Ling et al., 2025] — a long-context mathematical reasoning dataset with prompts up to 8,192 tokens and response lengths up to 2,048 tokens — using both GRPO and DAPO [Yu et al., 2025] (which removes the KL reference pass).

Setup. We train Qwen3-8B on a single `p5.48xlarge` instance (8×H100-SXM5-80GB) with FSDP2 in BF16 and gradient checkpointing. Rollout generation uses vLLM with tensor parallelism = 2 and N=32 responses per prompt. Training uses `train_batch_size` = 128 and `ppo_mini_batch_size` = 64 (both fixed across all configurations), and we sweep `ppo_micro_batch_size_per_gpu` $\in \{4, 8\}$. Each configuration runs for 25 training steps with validation every 2 steps. Although the rollout factor is N=32, the token reduction ratio ρ (Eq. 36) is bounded by the *micro-batch* size — only sequences co-located in the same forward/backward pass share a prompt. With P=8192, R=2048: $\rho = 2.5\times$ at mb=4 and $\rho = 3.3\times$ at mb=8, setting upper bounds on the measured policy-update speedup.

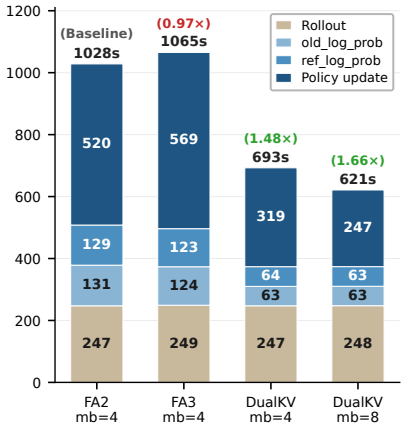

Figure 1: Per-phase step breakdown.

Comparisons. We compare four configurations (`mb` = per-GPU micro-batch size): FA2 (mb=4), the largest micro-batch FA2 can handle (mb=8 OOMs); FA3 (mb=4), evaluating the Hopper-specific kernel; DualKV (mb=4), an apple-to-apple comparison; and DualKV (mb=8), a configuration FA2 cannot run.

Results. Table 3 and Figure 2 summarize both experiments. All configs track identical training accuracy curves (Figure 2A), confirming DualKV introduces no convergence degradation for either method (formal proof in Appendix B).

Speedups, memory, and MFU. At mb=4, DualKV cuts policy-update time by $1.63\times$ (GRPO) and $1.91\times$ (DAPO), while reducing peak memory from 106 GB to 81 GB — a 25 GB reduction that brings the workload within the H100's 80 GB physical HBM. FA2's 106 GB allocation exceeds HBM capacity via memory oversubscription, degrading throughput. Doubling to mb=8 — which FA2 OOMs on — extends the speedup to $2.09\times$ (GRPO) and $2.47\times$ (DAPO), halving the gradient-accumulation passes per mini-batch ($2 \rightarrow 1$). Policy-update MFU rises from $36\%/31\%$ (FA2) to $76\%/77\%$ (DualKV mb=8) for GRPO/DAPO respectively — a 2.1–$2.5\times$ utilization gain. DAPO's higher speedups reflect the absence of the ref phase: the policy update dominates a larger fraction of step time, so DualKV's per-phase compression translates more directly to wall-clock gains.

End-to-end step speedup. Total training step time drops $1.64\times/1.82\times$ (GRPO/DAPO) at mb=8. The remaining gap versus the per-phase speedups is explained by rollout generation ($\sim$240 s/step, unaffected by DualKV), whose share of step time grows as training phases accelerate — DualKV shifts the bottleneck from the policy update to rollout generation (Figure 1). Full per-phase DAPO breakdowns are in Appendix E.2.

Table 3: End-to-end LongReason results. `Step` = total training-step time (s). `MFU` = policy-update Model FLOPs Utilization. Parentheses = speedup vs FA2 mb=4. Per-phase breakdown is in Figure 1.

Config	mb/GPU	Peak Mem (GB)	Step (s)	MFU (%)	Val Acc (step 24)
FA2	4	106†	1017 (1.00×)	35.8	0.777
FA3	4	106†	1067 (0.95×)	32.8	0.793
DualKV	4	81	687 (1.48×)	58.8	0.780
DualKV	8	93	622 (1.64×)	75.8	0.780
FA2	8	OOM during `loss.backward()`			

†PyTorch allocator reservation; actual HBM use stays within 80 GB.

4.3 Multi-Node RL Training with MoE Models

To validate DualKV at production scale with cross-node communication and MoE expert routing, we train Qwen3-30B-A3B [Yang et al., 2025] with GRPO on two `p5.48xlarge` nodes (16×H100 GPUs, Elastic Fabric Adapter): FSDP2 + BF16 + gradient checkpointing; rollout via vLLM (TP=2, N=32). All runs: `train_batch_size`=128, `ppo_mini_batch_size`=64, `ppo_micro_batch_size_per_gpu`=8, $P_{\max}$=8192, $R_{\max}$=2048. Timing is mean over 29 steps. We compare three configurations: DualKV (SP=1), no sequence parallelism; FA2 (SP=2), which OOMs during the first policy update; and FA2 (SP=4), the minimum SP that avoids OOM.

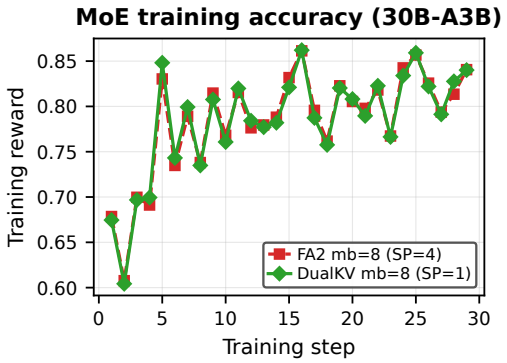

Figure 3: Per-step training accuracy.

Results. Table 4 summarizes the three configurations. DualKV delivers a $3.82\times$ policy-update speedup and $3.38\times$ total step speedup over FA2 SP=4, without requiring sequence parallelism. Both configs track identical training accuracy curves (Figure 3), confirming DualKV introduces no convergence degradation at multi-node training scale (formal proof in Appendix B).

Table 4: Wall-clock values are means over 29 steps; parenthesized values are speedups relative to FA2 SP=4.

Config	mb/GPU	SP	Peak Mem (GB)	Policy Update (s)	old_log_prob (s)	ref_log_prob (s)	Rollout (s)	Step (s)	Val Acc (step 28)
DualKV	8	1	103†	1078 (3.82×)	129 (3.45×)	142 (3.58×)	215 (1.00×)	1564 (3.38×)	0.774
FA2	8	4	92†	4115 (1.00×)	447 (1.00×)	507 (1.00×)	214 (1.00×)	5284 (1.00×)	0.802
FA2	8	2	OOM during first `loss.backward()`						

†PyTorch allocator reservation; actual HBM use stays within 80 GB.

Speedups at multi-node MoE scale. DualKV delivers a $3.82\times$ policy-update speedup and $3.38\times$ total-step speedup (5284 s $\rightarrow$ 1564 s, throughput 293 $\rightarrow$ 991 tokens/s). The larger gains versus the 8B single-node range (1.63–$2.09\times$) arise because (i) MoE expert GEMMs amplify the per-token savings from DualKV's $(N-1) \cdot P$ reduction, and (ii) DualKV eliminates the all-to-all communication that

FA2 with SP=4 pays on every attention call. Over 29 steps, DualKV completes in 12.6 hours vs. 42.6 hours for FA2 SP=4 on the same 16-GPU cluster (∼$6K saved at AWS `p5.48xlarge` on-demand list price). Figure 9 (Appendix F) visualizes the phase-wise breakdown.

Eliminating sequence parallelism. DualKV's memory reduction enables training at SP=1, whereas FA2 OOMs at SP=2 and requires SP=4. This translates memory savings into additional speedup: DualKV avoids the per-call all-to-all communication that SP imposes, which is why end-to-end gains ($3.38\times$) exceed what token dedup alone would predict.

4.4 Memory Scaling with Context Length

To characterize how DualKV's advantage scales with prompt length, we sweep $P \in \{8\text{K}–96\text{K}\}$ and mb $\in \{4, 8, 16\}$ on Llama-3.1-8B ($16\times$H100). This experiment also validates that DualKV generalizes across model families (Llama here versus Qwen3 in Sections 4.2–4.3).

Memory decouples from micro-batch size. DualKV's memory follows $M_0 + c_P \cdot P + c_{\text{mb}R} \cdot \text{mb} \cdot R$: the prompt cost is paid once regardless of micro-batch size, so the three mb curves cluster tightly (Figure 4). FA2's memory scales as $M_0 + c \cdot \text{mb} \cdot (P+R)$, diverging rapidly — at P=96K, increasing mb from 4 to 16 adds only $\sim$4 GB under DualKV ($< 5\%$), whereas FA2 would require $225 \rightarrow 775$ GB — nearly $10\times$ the physical capacity of an H100. Practitioners can freely increase mb to maximize throughput without memory penalty in the long-prompt regime.

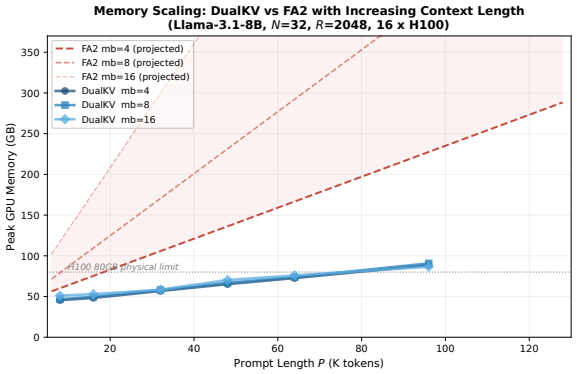

Figure 4: Memory scaling: DualKV vs. FA2.

4.5 DualKV with Sequence Parallelism on Gemma-4-31B

To show DualKV keeps pace with modern model architectures, we validate it end-to-end on Gemma-4-31B — a hybrid stack of interleaved head-dim-512 global and head-dim-256 sliding-window layers that standard FA2 cannot support — exercising both the hd512/SWA kernel (Section 3, Appendix I) and the SP composition (Section 2.3). Runs use LongReason (thinking enabled), N=16 rollouts, three epochs, on a single $8\times$H200 node; SP shards the actor and reference forward/backward. Table 5 reports the configurations we ran. First-token log-probs agree with the SP=1 computation to $\max|\Delta|=0$ across SP$\in\{1, 2, 4, 8\}$, and held-out accuracy remained stable with no degradation across all four configurations.

At 16K context, SP=4 lowers per-GPU peak memory from 96.8 to 77.0 GB while adding $\sim$12% step time — the expected cost of the two per-layer all-to-alls, spread across the three DualKV-accelerated training phases. At 32K context, where the deduplicated sequence is larger, SP=4 keeps peak memory flat at 79.8 GB (well within the 140 GB device) and completes full three-epoch training, demonstrating that DualKV composes with SP to reach contexts beyond a single rank's capacity while retaining the shared-prompt memory savings. At 64K with SP=8, peak memory stays flat at 78.9 GB — comparable to the 16K and 32K runs — since SP shards the sequence so the per-rank workload stays bounded as context grows; larger SP degrees follow by the same construction. DualKV's DAPO support (Section 4.2) and Gemma-4 support are orthogonal — both leave the shared-prompt attention and packing path unchanged — so they compose without additional kernel work. The hd512/SWA kernels, the SP composition, and the Gemma-4 run scripts are released on the `gemma4-dev` branch of the open-source repository: https://github.com/amazon-science/dualkv-flash-attn-for-rl/tree/gemma4-dev.

5 Related Work

Efficient attention for long sequences. FlashAttention [Dao et al., 2022, Dao, 2024, Shah et al., 2024] reduces attention memory from $O(S^2)$ to $O(S)$ via tiling and online softmax; Ring atten-

Table 5: DualKV with Ulysses SP on Gemma-4-31B (LongReason, N=16, thinking enabled, three epochs; single 8×H200 node). Wall-clock values are per-step means; peak memory is `max_memory_allocated`. SP=1 and SP=4 at 16K are separate runs of identical config (no offload, `gpu_mem`=0.4). Rollout generation (vLLM, SP-independent) is the remainder to the step total and is omitted. Held-out accuracy was stable with no degradation in all rows.

Config	Context	SP	Peak Mem (GB)	old_log_prob (s)	ref_log_prob (s)	Policy Update (s)
DualKV	16K	1	96.8	140	140	549
DualKV	16K	4	77.0	157	178	638
DualKV	32K	4	79.8	315	319	1242
DualKV	64K	8	78.9	782	791	3073

tion [Liu et al., 2024a], Ulysses [Jacobs et al., 2023], Megatron-SP [Korthikanti et al., 2023], and Context Parallelism [Grattafiori et al., 2024] distribute sequences across GPUs via sequence parallelism. All are orthogonal to DualKV's within-GPU shared-prompt deduplication and compose with it.

Shared-prompt optimization. Paged attention [Kwon et al., 2023], RadixAttention [Zheng et al., 2024], and bifurcated attention [Athiwaratkun et al., 2024] share prompt KV for inference only. Prefix Grouper [Liu et al., 2025] is a framework-level data reorganization that applies the shared-prompt decomposition to RL policy updates but replicates K, V N times before the attention call, retaining $O(N \cdot P \cdot d)$ memory at the kernel level. DualKV contributes the kernel-level primitive that eliminates this replication: a FlashAttention variant that reads shared KV from a single physical buffer and accumulates gradients from N concurrent sequences via fp32 atomic writes, fully realizing the decomposition's memory and compute savings for training (Appendix E.4 shows DualKV is 2–4× faster and uses 3–5× less memory than Prefix Grouper).

RL training systems. veRL [Sheng et al., 2025], OpenRLHF [Hu et al., 2024], and TRL [von Werra et al., 2020] implement PPO [Schulman et al., 2017]/GRPO [Shao et al., 2024]/REINFORCE++ [Hu et al., 2025]/DAPO [Yu et al., 2025] policy updates by packing N sequences with replicated prompts, without exploiting shared-prompt structure. DualKV is a drop-in replacement for the attention layer in these systems, requiring only data pipeline changes to group same-prompt responses.

6 Conclusion

DualKV eliminates the redundant prompt replication in RL policy updates by introducing a two-region FlashAttention kernel that reads shared KV from a single physical buffer, paired with a data-pipeline redesign that extends the token reduction to the entire model with provable gradient equivalence. Across dense and MoE models, DualKV achieves 2–3.8× policy-update speedup, raises MFU from 36% to 76%, and greatly reduces the need for sequence parallelism at long contexts. DualKV shifts the RL training bottleneck from the policy update to rollout generation. The current design assumes a single shared prefix per micro-batch group and benefits scale with P/R. Generalizing the kernel to arbitrary tree structures with shared sub-graphs (e.g., multi-turn conversations with partially overlapping prefixes or tree-of-thought search) would lift the single-prefix constraint. Because DualKV and sequence parallelism are orthogonal, DualKV's shared-prompt compression delays — and in many cases eliminates — the need for SP; for large models at ultra-long context, we compose DualKV with Ulysses SP and validate the combination on Gemma-4-31B at 64K context (Section 4.5), extending DualKV to modern hybrid attention architectures. Remaining future directions include porting to FA3/Hopper and FA4/Blackwell backends. Our implementation is open-sourced at https://github.com/amazon-science/dualkv-flash-attn-for-rl.

References

Ben Athiwaratkun, Sujan Kumar Gonugondla, Sanjay Krishna Gouda, Haifeng Qian, Hantian Ding, Qing Sun, Jun Wang, Jiacheng Guo, Liangfu Chen, Parminder Bhatia, Ramesh Nallapati, Sudipta Sengupta, and Bing Xiang. Bifurcated attention: Accelerating massively parallel decoding with shared prefixes in LLMs. *arXiv preprint arXiv:2403.08845*, 2024.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359, 2022. doi: 10.52202/068431-1189.

Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Jian Hu, Xibin Wu, Wei Shen, Jason Klein Liu, Zilin Zhu, Weixun Wang, Songlin Jiang, Haoran Wang, Hao Chen, Bin Chen, Weikai Fang, Xianyu, Yu Cao, Haotian Xu, and Yiming Liu. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. *arXiv preprint arXiv:2405.11143*, 2024.

Jian Hu, Jason Klein Liu, Haotian Xu, and Wei Shen. REINFORCE++: Stabilizing critic-free policy optimization with global advantage normalization. *arXiv preprint arXiv:2501.03262*, 2025.

Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. DeepSpeed Ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509*, 2023.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. Reducing activation recomputation in large transformer models. In *Proceedings of Machine Learning and Systems*, volume 5, pages 341–353, 2023.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023. doi: 10.1145/3600006.3613165.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, 2024.

Zhan Ling, Kang Liu, Kai Yan, Yifan Yang, Weijian Lin, Ting-Han Fan, Lingfeng Shen, Zhengyin Du, and Jiecao Chen. LongReason: A synthetic long-context reasoning benchmark via context expansion. *arXiv preprint arXiv:2501.15089*, 2025.

Hao Liu, Matei Zaharia, and Pieter Abbeel. RingAttention with blockwise transformers for near-infinite context. In *International Conference on Learning Representations*, 2024a.

Tianyang Liu, Canwen Xu, and Julian McAuley. RepoBench: Benchmarking repository-level code auto-completion systems. In *International Conference on Learning Representations*, 2024b.

Zikang Liu, Tongtian Yue, Yepeng Tang, Longteng Guo, Junxian Cai, Qingbin Liu, Xi Chen, and Jing Liu. Prefix grouper: Efficient GRPO training through shared-prefix forward. *arXiv preprint arXiv:2506.05433*, 2025.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems*, volume 37, pages 68658–68685. Curran Associates, Inc., 2024. doi: 10.52202/079017-2193.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297. Association for Computing Machinery, 2025. doi: 10.1145/3689031.3696075.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformers reinforcement learning. GitHub repository, 2020. URL `https://github.com/huggingface/trl`.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, volume 37, pages 62557–62583, 2024.

A DualKV Formal Derivation

This appendix provides a self-contained mathematical derivation of the DualKV forward and backward passes and proves equivalence to the FA2 baseline.

A.1 Setup and Notation

Consider N sequences sharing a prompt of P tokens. Sequence i has R_i response tokens, with total length $S_i = P + R_i$. Let $T = P + \sum_{i=1}^{N} R_i$ denote the total packed token count.

In a standard transformer layer, let $h \in \mathbb{R}^{T \times D}$ denote the hidden states after the input norm. The QKV projection produces:

$$Q = hW_Q, \quad K = hW_K, \quad V = hW_V \tag{9}$$

where $W_Q \in \mathbb{R}^{D \times Hd}$, $W_K \in \mathbb{R}^{D \times H_k d}$, $W_V \in \mathbb{R}^{D \times H_k d}$, with H query heads, H_k KV heads, and head dimension d.

A.2 Prompt Hidden State Invariant

Claim 1 (Prompt Hidden State Invariant)**.** *In a decoder-only model, for any prompt position $j < P$, the hidden state h_j is identical across all N sequences at every layer.*

Proof. By induction on layers. At the embedding layer, prompt tokens are shared, so $h_j^{(i)} = h_j$ for all i. Suppose the invariant holds at the input to layer ℓ. Every operation in the layer preserves it:

- **RMSNorm**: per-token, so $\text{Norm}(h_j^{(i)}) = \text{Norm}(h_j)$ for all i.
- **QKV projection**: linear per-token, so $Q_j^{(i)} = Q_j$, $K_j^{(i)} = K_j$, $V_j^{(i)} = V_j$.
- **RoPE**: depends only on position index j, applied per-token.

- **Causal attention**: for $j < P$, the causal mask restricts attention to keys at positions $\{0, \ldots, j\}$, all prompt positions with identical K and V. Combined with $Q_j^{(i)} = Q_j$ from the QKV-projection step, the attention output satisfies $O_j^{(i)} = O_j$ for all i.
- **Output projection, residual, MLP**: all per-token or element-wise.

The invariant holds at the output of layer ℓ, completing the induction. □

A.3 Forward Pass

Packing. We represent the micro-batch as a single packed sequence:

$$\mathbf{x} = [\underbrace{p_0, \ldots, p_{P-1}}_{\text{prompt}}, \underbrace{r_0^{(1)}, \ldots, r_{R_1-1}^{(1)}}_{\text{response 1}}, \ \ldots, \ \underbrace{r_0^{(N)}, \ldots, r_{R_N-1}^{(N)}}_{\text{response } N}] \tag{10}$$

All per-token operations process $T = P + \sum_i R_i$ tokens, saving $(N-1) \cdot P$ tokens versus the standard $\sum_i S_i = NP + \sum_i R_i$ layout.

QKV split. After the QKV projection on T tokens, we split at the prompt boundary P (zero-copy tensor views):

$$Q_c = Q_{[0:P]}, \quad K_c = K_{[0:P]}, \quad V_c = V_{[0:P]} \tag{11}$$

$$Q_d = Q_{[P:T]}, \quad K_d = K_{[P:T]}, \quad V_d = V_{[P:T]} \tag{12}$$

Call 1: Context self-attention. Standard causal attention over the P prompt tokens, computed once:

$$O_c = \text{CausalAttn}(Q_c, K_c, V_c) \in \mathbb{R}^{P \times H \times d} \tag{13}$$

Call 2: Decoded attention (DualKV kernel). For sequence i and decoded position $r \in \{0, \ldots, R_i - 1\}$, the query attends to the full key-value sequence $[K_c; K_d^{(i)}]$ with causal masking at logical position $P + r$:

$$O_d^{(i)}[r] = \sum_{j=0}^{P+r} \alpha_j^{(i,r)} \cdot v_j \tag{14}$$

where v_j indexes into $[V_c; V_d^{(i)}]$ and the attention weights are:

$$\alpha_j^{(i,r)} = \frac{\exp\big(q_r^{(i)} \cdot k_j / \sqrt{d}\big)}{\sum_{j'=0}^{P+r} \exp\big(q_r^{(i)} \cdot k_{j'} / \sqrt{d}\big)} \tag{15}$$

The `context_seqlen` $= P$ parameter shifts the kernel's internal position counter so that decoded query at kernel index r maps to logical position $P + r$, ensuring:

- All P context positions are unmasked ($j < P \leq P + r$).
- Decoded positions are causally masked ($j \leq P + r$).

Reassembly. The final attention output is:

$$O = [O_c; O_d] \in \mathbb{R}^{T \times H \times d} \tag{16}$$

which flows through W_O, residual addition, and MLP on T tokens.

A.4 Backward Pass

Let $\mathcal{L}$ denote the training loss (computed only over response tokens). We receive $\partial \mathcal{L} / \partial O \in \mathbb{R}^{T \times H \times d}$ and split it:

$$\frac{\partial \mathcal{L}}{\partial O_c} = \left(\frac{\partial \mathcal{L}}{\partial O}\right)_{[0:P]}, \quad \frac{\partial \mathcal{L}}{\partial O_d} = \left(\frac{\partial \mathcal{L}}{\partial O}\right)_{[P:T]} \tag{17}$$

Note that $\partial \mathcal{L} / \partial O_c \neq 0$ in general: even though the loss is on response tokens only, gradients propagate back to prompt positions through subsequent layers' residual connections.

Call 2 backward (DualKV kernel). Given $\partial\mathcal{L}/\partial O_d$ and the saved tensors from the forward pass, the backward produces:

$$\frac{\partial\mathcal{L}}{\partial Q_d}, \quad \frac{\partial\mathcal{L}}{\partial K_d}, \quad \frac{\partial\mathcal{L}}{\partial V_d} \tag{18}$$

and gradients with respect to the shared context KV:

$$\left(\frac{\partial\mathcal{L}}{\partial K_c}\right)_d = \sum_{i=1}^{N}\sum_{r=0}^{R_i-1}\sum_h \frac{\partial\mathcal{L}}{\partial O_d^{(i)}[r,h]} \cdot \frac{\partial O_d^{(i)}[r,h]}{\partial K_c} \tag{19}$$

$$\left(\frac{\partial\mathcal{L}}{\partial V_c}\right)_d = \sum_{i=1}^{N}\sum_{r=0}^{R_i-1}\sum_h \frac{\partial\mathcal{L}}{\partial O_d^{(i)}[r,h]} \cdot \frac{\partial O_d^{(i)}[r,h]}{\partial V_c} \tag{20}$$

Implementation: fp32 gradient accumulation. The summation over N sequences in Eq. 19–20 introduces a concurrent-write pattern absent from standard FlashAttention: the single shared dK_c and dV_c buffer has no batch offset, yet N thread blocks (one per decoded sequence) concurrently compute partial contributions to the same context positions. DualKV resolves this with fp32 atomic accumulation in a dedicated accumulator buffer, followed by a cast kernel that converts the accumulator to bf16. The kernel-level description and pseudocode are given in Section 3.2 (Algorithms 1–2). For the purposes of the mathematical derivation that follows, the relevant property is that the kernel produces $(\partial\mathcal{L}/\partial K_c)_d = \sum_i \sum_r \sum_h \partial\mathcal{L}/\partial O_d^{(i)}[r,h] \cdot \partial O_d^{(i)}[r,h]/\partial K_c$ exactly (up to fp32 rounding) as stated in Eq. 19–20.

Call 1 backward (standard FA2). Given $\partial\mathcal{L}/\partial O_c$, produces:

$$\frac{\partial\mathcal{L}}{\partial Q_c}, \quad \left(\frac{\partial\mathcal{L}}{\partial K_c}\right)_c, \quad \left(\frac{\partial\mathcal{L}}{\partial V_c}\right)_c \tag{21}$$

Gradient accumulation. Since K_c and V_c are the *same tensor objects* passed to both Call 1 and Call 2, PyTorch autograd accumulates their gradients:

$$\frac{\partial\mathcal{L}}{\partial K_c} = \left(\frac{\partial\mathcal{L}}{\partial K_c}\right)_c + \left(\frac{\partial\mathcal{L}}{\partial K_c}\right)_d \tag{22}$$

$$\frac{\partial\mathcal{L}}{\partial V_c} = \left(\frac{\partial\mathcal{L}}{\partial V_c}\right)_c + \left(\frac{\partial\mathcal{L}}{\partial V_c}\right)_d \tag{23}$$

The full QKV gradients are then reassembled:

$$\frac{\partial\mathcal{L}}{\partial Q} = \left[\frac{\partial\mathcal{L}}{\partial Q_c}; \frac{\partial\mathcal{L}}{\partial Q_d}\right], \quad \frac{\partial\mathcal{L}}{\partial K} = \left[\frac{\partial\mathcal{L}}{\partial K_c}; \frac{\partial\mathcal{L}}{\partial K_d}\right], \quad \frac{\partial\mathcal{L}}{\partial V} = \left[\frac{\partial\mathcal{L}}{\partial V_c}; \frac{\partial\mathcal{L}}{\partial V_d}\right] \tag{24}$$

These propagate through the QKV projection backward to produce $\partial\mathcal{L}/\partial h$ and $\partial\mathcal{L}/\partial W_Q$, $\partial\mathcal{L}/\partial W_K$, $\partial\mathcal{L}/\partial W_V$.

A.5 Equivalence to the FA2 Baseline

Theorem 1 (Gradient Equivalence)**.** *The parameter gradients produced by DualKV are identical to those produced by the FA2 baseline.*

Proof. In the standard approach, each sequence $i \in \{1, \ldots, N\}$ has its own copy of the prompt KV: $K_c^{(i)}, V_c^{(i)}$. The gradient of $K_c^{(i)}$ has two components:

$$\frac{\partial\mathcal{L}}{\partial K_c^{(i)}} = \underbrace{g_{\text{self}}^{(i)}}_{\text{prompt self-attn}} + \underbrace{g_{\text{cross}}^{(i)}}_{\text{decoded}\to\text{context}} \tag{25}$$

The parameter gradient from the W_K projection at prompt positions is:

$$\left.\frac{\partial\mathcal{L}}{\partial W_K}\right|_{\text{prompt}} = \sum_{i=1}^{N}\left(\frac{\partial\mathcal{L}}{\partial K_c^{(i)}}\right)^\top h_c = \left(\sum_{i=1}^{N}\frac{\partial\mathcal{L}}{\partial K_c^{(i)}}\right)^\top h_c \tag{26}$$

where h_c is the prompt hidden state (identical across i by the invariant, so it factors out of the sum).

In DualKV, the single K_c receives gradient (Eq. 22):

$$\frac{\partial \mathcal{L}}{\partial K_c} = \left(\frac{\partial \mathcal{L}}{\partial K_c}\right)_c + \sum_{i=1}^{N} g_{\text{cross}}^{(i)} \tag{27}$$

The parameter gradient is:

$$\frac{\partial \mathcal{L}}{\partial W_K}\bigg|_{\text{prompt}}^{\text{DualKV}} = \left(\frac{\partial \mathcal{L}}{\partial K_c}\right)^{\top} h_c \tag{28}$$

It remains to show that Eq. 26 equals Eq. 28, i.e.:

$$\sum_{i=1}^{N} \left(g_{\text{self}}^{(i)} + g_{\text{cross}}^{(i)}\right) = \left(\frac{\partial \mathcal{L}}{\partial K_c}\right)_c + \sum_{i=1}^{N} g_{\text{cross}}^{(i)} \tag{29}$$

This reduces to showing:

$$\sum_{i=1}^{N} g_{\text{self}}^{(i)} = \left(\frac{\partial \mathcal{L}}{\partial K_c}\right)_c \tag{30}$$

In the standard approach, $g_{\text{self}}^{(i)}$ is the gradient of $K_c^{(i)}$ from the context self-attention of sequence i. This depends on $\partial \mathcal{L}/\partial O_c^{(i)}$, which propagates back from subsequent layers. While the forward values at prompt positions are identical across i, the backward gradients $\partial \mathcal{L}/\partial O_c^{(i)}$ generally differ (each sequence i has different response tokens producing different loss contributions).

In DualKV, the single prompt copy receives the *aggregated* upstream gradient:

$$\frac{\partial \mathcal{L}}{\partial O_c} = \sum_{i=1}^{N} \frac{\partial \mathcal{L}}{\partial O_c^{(i)}} \tag{31}$$

This aggregation happens naturally through the residual stream: in the packed layout, prompt hidden states at layer $\ell{+}1$ receive gradients from all N response branches propagating backward through subsequent layers. Since all intervening operations (norms, projections, MLP, residual) are per-token and *linear in the backward pass*, the gradient at the prompt positions is exactly the sum over all N sequences' contributions.

The context self-attention backward is linear in $\partial \mathcal{L}/\partial O_c$:

$$\left(\frac{\partial \mathcal{L}}{\partial K_c}\right)_c = f_{\text{attn-bwd}}\left(\frac{\partial \mathcal{L}}{\partial O_c}\right) = f_{\text{attn-bwd}}\left(\sum_{i=1}^{N} \frac{\partial \mathcal{L}}{\partial O_c^{(i)}}\right) = \sum_{i=1}^{N} f_{\text{attn-bwd}}\left(\frac{\partial \mathcal{L}}{\partial O_c^{(i)}}\right) = \sum_{i=1}^{N} g_{\text{self}}^{(i)} \tag{32}$$

where $f_{\text{attn-bwd}}$ denotes the attention backward function mapping dO to dK, which is linear in dO (since the forward values Q_c, K_c, V_c are fixed during backward). This establishes Eq. 30 and completes the proof. □

A.6 Complexity Summary

This subsection summarizes the per-component FLOPs and memory scaling of FA2 baseline vs. DualKV in asymptotic notation, abstracting the concrete numbers of Section 2. The table shows that the savings are dominated by the token-count reduction $T_{\text{std}} \to T_{\text{dk}}$: every per-token cost (norms, projections, MLP, KV storage, activation memory) shrinks by the ratio $\rho = T_{\text{std}}/T_{\text{dk}}$, and only attention has additional structural savings beyond that.

When $P \gg R$ and $R_i = R$ for all i:

$$\frac{T_{\text{std}}}{T_{\text{dk}}} = \frac{NP + NR}{P + NR} \xrightarrow{P \gg R} N \tag{33}$$

The savings approach a factor of N (the batch size) across all components — norms, projections, MLP, attention, and activation memory — yielding near-linear scaling.

Table 6: Complexity comparison. $T_{\text{std}} = NP + \sum_i R_i$, $T_{\text{dk}} = P + \sum_i R_i$.

Component	FA2 baseline	DualKV
Norms, projections, MLP	$O(T_{\text{std}} \cdot D^2)$	$O(T_{\text{dk}} \cdot D^2)$
Attention FLOPs	$\sum_i O(S_i^2 \cdot H \cdot d)$	$O(P^2 H d) + \sum_i O(R_i S_i H d)$
KV memory per layer	$O(T_{\text{std}} \cdot H_k \cdot d)$	$O(T_{\text{dk}} \cdot H_k \cdot d)$
Activation memory	$O(T_{\text{std}} \cdot D)$	$O(T_{\text{dk}} \cdot D)$

B Training-Step Gradient Equivalence

Appendix A proves the DualKV kernel produces attention gradients identical to the FA2 baseline in exact arithmetic for any fixed packed input. This appendix lifts that kernel-level invariance to the training-step level. DualKV introduces two modifications to veRL's policy-update data pipeline, both motivated by the kernel's same-prompt-per-micro-batch packing contract:

1. **Skip** `balance_batch`**.** veRL's default pipeline reorders the rollout list by per-sample sequence length to equalize per-GPU token counts before dispatch. DualKV skips this step, since the seqlen permutation scatters rollouts from the same prompt across different GPUs, breaking the single-prompt packing the kernel requires. The global rollout multiset is unchanged; only its ordering and per-GPU partition differ.
2. **Force** `shuffle=False` within the per-rank mini-batch iterator. Under the default pipeline, samples within each rank are reshuffled between PPO epochs. DualKV disables this to preserve contiguous prompt-grouped ordering across epochs.

For one training step, let B be the global multiset of (p, r, A) triples produced by that step's rollout, where p is a prompt, r a sampled response, and $A = A(p, r)$ a scalar advantage. The policy-gradient estimator is $\hat{g}(B, \theta) = \sum_{(p,r,A)\in B} f(p, r, A, \theta)$, covering standard RL objectives (PPO, GRPO, DAPO). Let $\mathcal{P}_{\text{def}}$ denote veRL's default pipeline and $\mathcal{P}_{\text{dk}}$ the DualKV pipeline.

Theorem 2 (No systematic bias at training-step aggregate)**.** *For any parameters θ,*

$$\sum_{(p,r,A)\in B} f^{\mathcal{P}_{def}}(p, r, A, \theta) \;=\; \sum_{(p,r,A)\in B} f^{\mathcal{P}_{dk}}(p, r, A, \theta) \qquad \textit{in exact arithmetic.} \tag{34}$$

Proof. The per-sample gradient $f(p, r, A, \theta)$ depends on θ, the advantage A, and the per-token log-probabilities $\pi_\theta(r \mid p)$. Advantages are computed upstream of the policy update and are pipeline-independent. The kernel-level equivalence theorem of Appendix A establishes the primary lemma we rely on: log-probabilities and their gradients at response positions are identical under $\mathcal{P}_{\text{def}}$ and $\mathcal{P}_{\text{dk}}$ for any fixed sample. Hence $f^{\mathcal{P}_{\text{def}}}(p, r, A, \theta) = f^{\mathcal{P}_{\text{dk}}}(p, r, A, \theta)$ termwise. The two pipeline modifications are both permutations of the rollout list, so the global multiset B is the same under both pipelines. The equation follows by termwise equality and order-invariance of finite sums. □

C Theoretical Speedup and Memory Analysis

This appendix derives per-layer kernel-level memory savings and the closed-form attention speedup ratio for DualKV. These are theoretical ceilings; measured end-to-end numbers are in Sections 4.2–4.3 (and Appendix E.1).

C.1 Memory Savings

Per layer, the DualKV kernel avoids storing $N-1$ redundant copies of three tensors that an equivalent N-copy FA2 call would materialize:

These are *kernel-level* savings: they count the per-layer attention-input tensors (Q at prompt positions, K_c, V_c) and the prompt-position `softmax_lse` that an FA2 baseline would materialize N times and DualKV materializes once. DualKV's single-prompt packing also reduces framework-level activation memory across every per-token operation (norms, QKV/output projections, MLP, residual stream), since these now process $P + NR$ tokens instead of $N(P + R)$.

Table 7: Kernel-level memory savings per layer relative to an N-copy FA2 call over the same packed batch. Configuration: Qwen3-8B (N=8, P=16384, R=2048, H=32, H_k=8, d=128) in bf16.

Tensor saved	**Per-layer savings (bytes)**	**Numerical**
K_c, V_c (shared prompt KV)	$2 \cdot (N-1) \cdot P \cdot H_k \cdot d \cdot 2$	469 MB
Q at prompt positions (avoided vs. FA2 baseline)	$(N-1) \cdot P \cdot H \cdot d \cdot 2$	940 MB
`softmax_lse` for prompt positions	$(N-1) \cdot P \cdot H \cdot 4$	14 MB
Total per layer		**1423 MB**

C.2 Speedup Scaling

The attention speedup ratio scales approximately as:

$$\text{Speedup}_{\text{attn}} \approx \frac{N \cdot S^2}{P^2 + N \cdot R \cdot S} = \frac{N(P+R)^2}{P^2 + NR(P+R)} \tag{35}$$

For $P \gg R$, this simplifies to $\approx N$, meaning DualKV approaches a linear speedup in batch size as prompt length dominates. For P=32K, R=2K, N=16: theoretical speedup is $6.1\times$, measured fwd+bwd is $5.48\times$ (90% efficiency). The consistent speedup across forward and backward passes confirms that the theoretical FLOPs analysis (Section 2) applies equally to both directions.

The end-to-end training speedup is lower than this kernel-level ceiling, as attention is only one component of the training step (alongside generation, projections, MLP, optimizer, and communication). The token reduction ratio ρ (Section 2) provides an upper bound on the full-model speedup.

C.3 Token Reduction Ratio ρ: Scaling Analysis and Regimes

The token reduction ratio of DualKV over standard attention is:

$$\rho = \frac{T_{\text{std}}}{T_{\text{dk}}} = \frac{N(P+R)}{P+NR} \tag{36}$$

This ratio governs the speedup across *all* model components (norms, projections, MLP, activation memory), not just attention. We analyze its behavior along the two key axes: rollout factor N and prompt-to-response ratio P/R.

Scaling with N (rollout factor). Fixing P and R, taking the limit:

$$\lim_{N\to\infty} \rho = \lim_{N\to\infty} \frac{N(P+R)}{P+NR} = \frac{P+R}{R} = 1 + \frac{P}{R} \tag{37}$$

For large N, the speedup saturates at $1 + P/R$. With P=16K, R=2K: ceiling is $9\times$. Increasing N beyond this point yields diminishing returns — the bottleneck shifts to decoded-token compute $O(NR)$ which scales linearly in both approaches.

Scaling with P/R (prompt length ratio). Fixing N and R, as $P \to \infty$:

$$\lim_{P\to\infty} \rho = \lim_{P\to\infty} \frac{NP+NR}{P+NR} = N \tag{38}$$

For long prompts, the speedup approaches N — near-linear in the rollout factor. This is the regime where DualKV is most impactful: the $(N-1)P$ redundant prompt tokens dominate total compute.

Concrete scenarios. Table 8 shows the token reduction ratio ρ for representative RL training configurations.

Two regimes emerge where DualKV provides substantial benefits:

1. **Large rollout factor** ($N \geq 16$): Common in GRPO and DAPO, where generating many candidate responses per prompt improves reward signal quality. At N=32 with P=16K, the reduction reaches $7.2\times$.

Table 8: Token reduction ratio $\rho = T_{\text{std}}/T_{\text{dk}}$ for various (N, P, R) configurations.

N	P	R	ρ	**Regime**
8	2K	2K	1.8×	Short prompt, moderate rollout
8	16K	2K	4.5×	Long prompt, moderate rollout
8	64K	512	7.6×	Very long prompt
16	16K	2K	6.0×	Long prompt, large rollout
32	4K	2K	2.8×	Short prompt, very large rollout
32	16K	2K	7.2×	Long prompt, very large rollout
16	32K	2K	8.5×	Very long prompt, large rollout
16	64K	512	14.3×	Ultra-long prompt, large rollout

2. **Long prompts** ($P \geq 16$K): Prevalent in agentic tasks, repository-level code generation, and multi-turn dialogue. At P=64K, even moderate rollout (N=8) yields 7.6× reduction, and N=16 reaches 14.3×.

D Software Environment

All kernel-level benchmarks (Section 4.1) and end-to-end GRPO training runs (Sections 4.2–4.3 and Appendix E.1) use an identical Python virtual environment. Configuration flags (`use_dualkv`, `ulysses_sequence_parallel_size`, `attn_implementation`) are the only variables that differ between runs.

Table 9: Software stack used across all experiments. "DualKV build" indicates our fork of `flash_attn` 2.8.4 with the DualKV kernel (Section 3) added; base FA2 behavior is unchanged. "DualKV integration" indicates our patch to `verl` 0.7.0 that adds the data-pipeline changes (Section 2.2) and actor/trainer hooks needed to use the DualKV kernel.

Package	**Version**
Python	3.12
`torch`	2.9.0+cu128
`vllm`	0.12.0
`flash_attn`	2.8.4 (DualKV build)
`verl`	0.7.0 (DualKV integration)
`transformers`	4.57.6
`ray`	2.55.1
NCCL	2.27.5

The veRL patch (∼1200 lines across 10 files) modifies the actor's forward pass, the trainer's data pipeline (skipping `balance_batch`, enforcing prompt-grouping), and a monkey-patch hook that installs the DualKV attention call on supported model classes. Both the `flash_attn` and `verl` patches are available at https://github.com/amazon-science/dualkv-flash-attn-for-rl.

E Additional Experiments

E.1 GRPO Training on GSM8K (Short Prompts)

This appendix presents our short-prompt validation experiment (referenced in Section 4 introduction). The experiment stress-tests DualKV in the low-ρ regime ($\rho = 1.8$–$3.8\times$), where kernel-level speedups are modest but memory savings still extend the feasible training frontier.

We evaluate DualKV in end-to-end GRPO training using the veRL framework on GSM8K [Cobbe et al., 2021], comparing against standard FA2 packing across a range of prompt lengths.

Setup. We train Qwen3-8B [Yang et al., 2025] with GRPO on an 8-node cluster of NVIDIA A100-SXM4-40GB GPUs (64 GPUs total). The model is sharded via FSDP across 32 data-parallel ranks (tensor parallelism = 2), with gradient checkpointing enabled and optimizer states offloaded to CPU. Each training step consists of: (1) rollout generation via vLLM (N=8 responses

per prompt, $R_{\max}$=256), (2) log-probability computation, and (3) policy update with gradient accumulation (micro-batch size $= 8$ per GPU). We use `ppo_mini_batch_size` $= 128$ and `train_batch_size` $= 512$.

Dataset. To control prompt length, we construct synthetic GSM8K variants by prepending few-shot examples to each prompt until the target token count is reached. We sweep prompt lengths $P \in \{1\text{K}, 1.5\text{K}, 2\text{K}, 2.5\text{K}, 3\text{K}\}$ tokens, all with R=256 and N=8. Each configuration runs 3 training steps; we report metrics from step 3 (after 2 warmup steps) to exclude one-time initialization costs.

Prompt grouping. DualKV requires all N responses from the same prompt to be co-located on the same GPU within each micro-batch. We modify veRL's data pipeline to (1) skip the default `balance_batch` reordering when DualKV is enabled, preserving prompt-group contiguity, and (2) add runtime assertions verifying that every micro-batch contains only same-prompt rollouts. The micro-batch size ($= 8$) equals N, ensuring each forward/backward pass processes exactly one prompt group. Appendix B proves that these data-pipeline modifications preserve the exact mini-batch gradient estimator $\hat{g}(B, \theta)$ under data-parallel training, so they do not alter training dynamics at any parameter setting.

Results. Table 10 reports peak GPU memory (reserved) and wall-clock time for both the full training step and the policy update phase (where DualKV applies). Rollout generation uses vLLM identically in both configurations and is excluded from the policy update timing.

Table 10: End-to-end GRPO training: FA2 vs. DualKV at micro-batch size 8, N=8, R=256. Qwen3-8B on 64×A100-40GB. "Update" = policy update time. OOM = out of memory during policy update.

		Peak Memory (GB)		**Step Time (s)**		**Update Time (s)**	
P	ρ	FA2	DualKV	FA2	DualKV	FA2	DualKV
1K	1.8×	37.8	**26.5**	457	**449**	175	**168**
1.5K	2.3×	44.4	**27.3**	499	**486**	177	**169**
2K	2.9×	46.8‡	**28.1**	570	**528**	203	**171**
2.5K	3.3×	OOM	**29.0**	—	584	—	173
3K	3.8×	OOM	**29.8**	—	618	—	176

$\rho = N(P+R)/(P+NR)$ is the token reduction ratio. ‡Exceeds A100-40GB physical memory; PyTorch uses unified memory with severe performance degradation.

Memory scaling. FA2 peak memory grows rapidly with prompt length (37.8→44.4→46.8 GB), exceeding the A100's 40 GB physical memory at P=1.5K and hitting OOM at P=2.5K. DualKV memory grows slowly (26.5→29.8 GB), remaining well within the 40 GB budget at all tested contexts. The memory gap widens from 11.3 GB (-30%) at P=1K to 18.7 GB (-40%) at P=2K. Beyond P=2K, FA2 cannot train at all at micro-batch size 8, while DualKV continues to scale.

Policy update speedup. The policy update phase (forward/backward passes with gradient accumulation) shows increasing speedup: $1.04\times$ at P=1K, $1.05\times$ at P=1.5K, and $1.19\times$ at P=2K. The update time for DualKV is nearly constant across prompt lengths (168–176 s), confirming that the shared prompt KV is processed only once regardless of prompt length.

End-to-end step time. Total step time improvement is more modest (1.02–$1.08\times$) because rollout generation (vLLM inference) and log-probability computation — which are identical for both methods — dominate. At P=2K, the policy update accounts for only 36% of FA2's step time but receives $1.19\times$ speedup, yielding an overall $1.08\times$ improvement.

Extending the training frontier. The most impactful result is that DualKV *enables training* at context lengths where FA2 cannot run. At P=3K with N=8 and micro-batch size 8, FA2 requires $\sim 8 \times (3072{+}256) = 26,624$ tokens per micro-batch forward pass, while DualKV requires only $3072 + 8 \times 256 = 5,120$ tokens — a $5.2\times$ reduction in activation memory. This enables practitioners to train with longer prompts on existing hardware without reducing batch size or adding gradient checkpointing overhead.

Numerical correctness. We verified that DualKV matches standard FA2 to within half-precision rounding in two independent tests: (1) kernel-level forward pass comparison using `torch.allclose`

(atol=10^{-3}, rtol=10^{-3} in fp16) across all supported head dimensions (64, 96, 128, 192, 256), GQA configurations, and context/decoded lengths; and (2) end-to-end inference with Qwen2-1.5B via vLLM, where DualKV produced *identical token sequences* to the standard attention baseline across 4 sequences sharing a common prompt (greedy decoding, 128 tokens). These results confirm that the two-phase attention decomposition and gradient accumulation introduce no numerical divergence beyond the expected half-precision rounding.

Note on ρ values. The token reduction ratios in this experiment ($\rho = 1.8$–$3.8\times$) are moderate because N=8 and R=256 (short responses). The kernel-level benchmarks in Table 1 use N=28 and R=2048, achieving ρ up to $7\times$ with correspondingly larger speedups. In production RL training with N=16–32 and longer responses, the end-to-end benefits will be substantially greater.

E.2 DAPO Training on LongReason

DualKV benefits any sampling-based RL method that packs N same-prompt responses into a micro-batch. To demonstrate this generality beyond GRPO, we repeat the LongReason experiment (Section 4.2) using DAPO [Yu et al., 2025] — a sampling-based RL method that removes the KL penalty (no reference model forward pass) and uses dynamic clipping with token-level loss aggregation.

Setup. Identical to Section 4.2: Qwen3-8B on a single `p5.48xlarge` instance (8×H100), FSDP2 + BF16 + gradient checkpointing, vLLM rollout (TP=2, N=32), `train_batch_size`=128, `ppo_mini_batch_size`=64, $P_{\max}$=8192, $R_{\max}$=2048, 25 training steps. DAPO-specific: `clip_ratio_low`=0.2, `clip_ratio_high`=0.28, `clip_ratio_c`=10.0, `loss_agg_mode`=token-mean, no KL loss. Because DAPO does not use a reference model, the `ref_log_prob` phase is absent; the training step consists of rollout, `old_log_prob`, and the policy update only.

Comparisons. Same four configurations as Section 4.2: FA2 (mb=4), FA3 (mb=4), DualKV (mb=4), and DualKV (mb=8).

Results. Table 11 summarizes the results. DualKV delivers $1.91\times$ (mb=4) and $2.47\times$ (mb=8) policy-update speedup over FA2, comparable to the GRPO speedups ($1.63\times$/$2.09\times$). The slightly higher speedups reflect the absence of the ref phase: the policy update's share of step time is larger in DAPO, so DualKV's per-phase compression translates more directly to end-to-end gains. Step speedup is $1.63\times$ (mb=4) and $1.82\times$ (mb=8). MFU rises from 30.9% (FA2) to 77.4% (DualKV mb=8) — a $2.5\times$ utilization gain. Peak memory drops from 106 GB to 80 GB (mb=4) and 93 GB (mb=8), confirming the same memory-headroom story. All four configs track identical training accuracy curves (final reward ≈ 0.85–0.87), confirming no convergence degradation.

Table 11: DAPO training on LongReason (Qwen3-8B, 8×H100). Same setup as Table 3 but with DAPO instead of GRPO. No `ref_log_prob` phase (DAPO removes the KL penalty). Speedups relative to FA2 mb=4.

Config	mb/GPU	Peak Mem (GB)	Policy Update (s)	old_log_prob (s)	ref_log_prob (s)	Rollout* (s)	Step (s)	MFU (%)	Reward (step 25)
FA2	4	106	597 (1.00×)	153 (1.00×)	—	236 (1.00×)	987 (1.00×)	30.9	0.848
FA3	4	106	543 (1.10×)	142 (1.08×)	—	234 (1.01×)	919 (1.07×)	34.0	0.859
DualKV	4	80	313 (1.91×)	63 (2.43×)	—	231 (1.02×)	606 (1.63×)	59.3	0.853
DualKV	8	93	242 (2.47×)	63 (2.43×)	—	237 (1.00×)	542 (1.82×)	77.4	0.868

*`Rollout` (vLLM inference) is not targeted by DualKV.

Comparison with GRPO. The per-phase speedups are consistent with Section 4.2: DualKV compresses `old_log_prob` at $\sim 2.4\times$ (vs $\sim 2.1\times$ in GRPO) and the policy update at 1.91–$2.47\times$ (vs 1.63–$2.09\times$). The slightly larger DAPO speedups are attributable to (i) absence of the ref phase reducing the denominator, and (ii) DAPO's token-level loss aggregation producing marginally different gradient-accumulation patterns that favor larger effective batch sizes. The key takeaway is that DualKV uniformly accelerates any sampling-based RL method that packs N same-prompt responses.

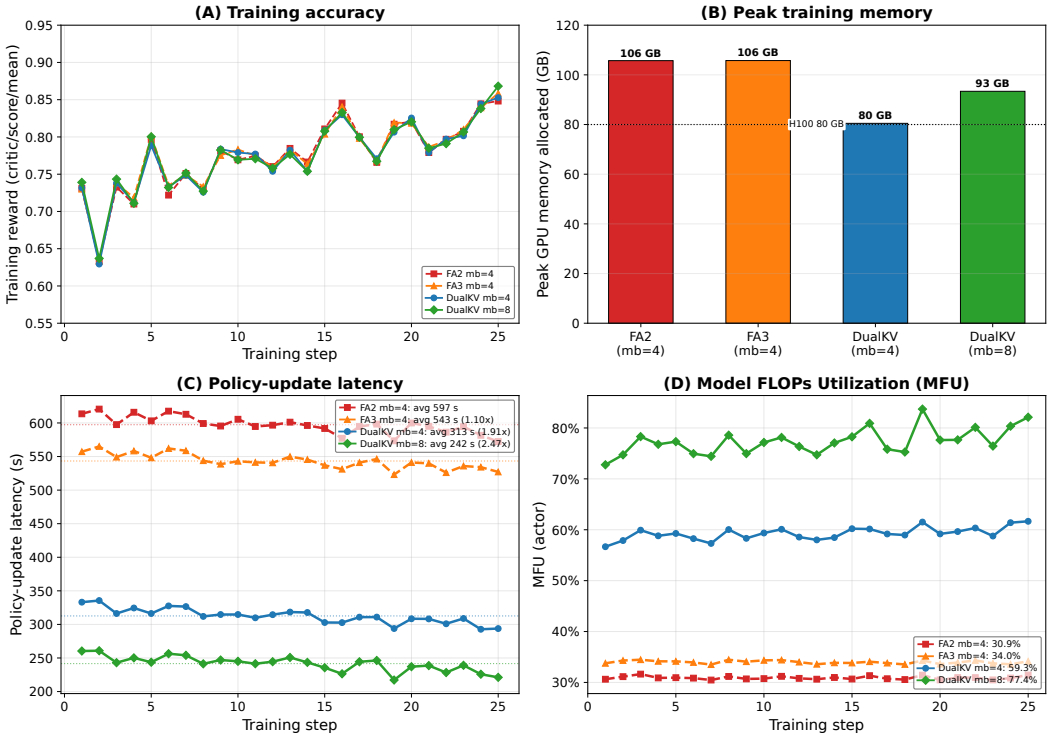

Figure 5: DAPO training on LongReason (Qwen3-8B, 8×H100). **(A)** All four configurations track identical reward curves, confirming no convergence degradation. **(B)** Peak memory: FA2/FA3 exceed the H100 80 GB physical limit (∼106 GB allocated), while DualKV mb=4 fits at ∼80 GB. **(C)** Policy-update latency: DualKV mb=8 achieves 2.47× speedup over FA2. **(D)** MFU rises from 31% (FA2) to 77% (DualKV mb=8).

E.3 Context-Length Scaling Sweep (Llama-3.1-8B)

To validate that DualKV's benefits generalize beyond Qwen and to empirically characterize memory scaling, we conduct a context-length sweep on Llama-3.1-8B-Instruct [Grattafiori et al., 2024] across prompt lengths $P \in \{8\text{K}, 16\text{K}, 32\text{K}, 48\text{K}, 64\text{K}, 96\text{K}\}$ and micro-batch sizes $\text{mb} \in \{4, 8, 16\}$.

Setup. Hardware: 2×`p5.48xlarge` (16×H100-80GB). GRPO with N=32, $R_{\max}$=2048, `train_batch_size`=128, vLLM rollout (TP=2), DualKV + FusedLinearForPPO enabled, `gpu_memory_utilization`=0.4.

Memory scaling law. Figure 6 plots peak GPU memory (`max_memory_allocated`) against prompt length for all 18 configurations. DualKV's memory follows:

$$\text{mem}_{\text{DualKV}} = \underbrace{M_0}_{\substack{\text{model + optim +} \\ \text{framework}}} + \underbrace{c_P \cdot P}_{\substack{\text{prompt activations} \\ \text{(paid once)}}} + \underbrace{c_{\text{mb}R} \cdot \text{mb} \cdot R}_{\substack{\text{response activations} \\ \text{(scales with mb)}}} \tag{39}$$

where the coefficients are estimated by least-squares regression across all 18 data points (6 prompt lengths × 3 micro-batch sizes): $M_0 \approx 41.1$ GB, $c_P \approx 0.475$ GB/Ktoken, $c_{\text{mb}R} \approx 0.122$ GB/Ktoken ($r^2 = 0.989$). The asymmetry $c_P \gg c_{\text{mb}R}$ arises because prompt tokens incur higher per-token activation cost under gradient checkpointing: block-boundary activations are retained for all P positions across all layers, and the context self-attention backward (Call 1) saves `softmax_lse` of size $O(P \times H)$. Response tokens in Call 2 attend only to their own shorter decoded subsequence, requiring less stored state. The key insight is structural: the prompt term $c_P \cdot P$ does not multiply by mb — DualKV processes the shared prompt once regardless of how many responses are in the micro-batch. Under standard FA2 packing, each of the mb sequences carries its own full copy of the prompt, so memory scales as:

$$\text{mem}_{\text{FA2}} = \underbrace{M_0}_{\text{same}} + \underbrace{c_P \cdot \text{mb} \cdot (P + R)}_{\text{prompt replicated mb times}} \tag{40}$$

making memory *linear in mb*. FA2 values in Figure 6 are projections from Eq. 40 (using the same fitted M_0 and c_P) rather than measurements, because FA2 triggers CUDA OOM at these context lengths — the standard mb $\times$ $(P + R)$ packing exceeds the H100's 80 GB HBM for most configurations in this sweep.

Practical implication. When $P \gg \text{mb} \cdot R$ (the long-prompt regime common in agentic and reasoning tasks), memory is dominated by the prompt term and becomes effectively independent of micro-batch size. Practitioners can freely increase mb to maximize GPU utilization without memory penalty: at P=96K, increasing mb from 4 to 16 adds only $\sim$4 GB ($< 5\%$), whereas FA2 would require 225 $\rightarrow$ 775 GB — nearly 10$\times$ the physical capacity of an H100.

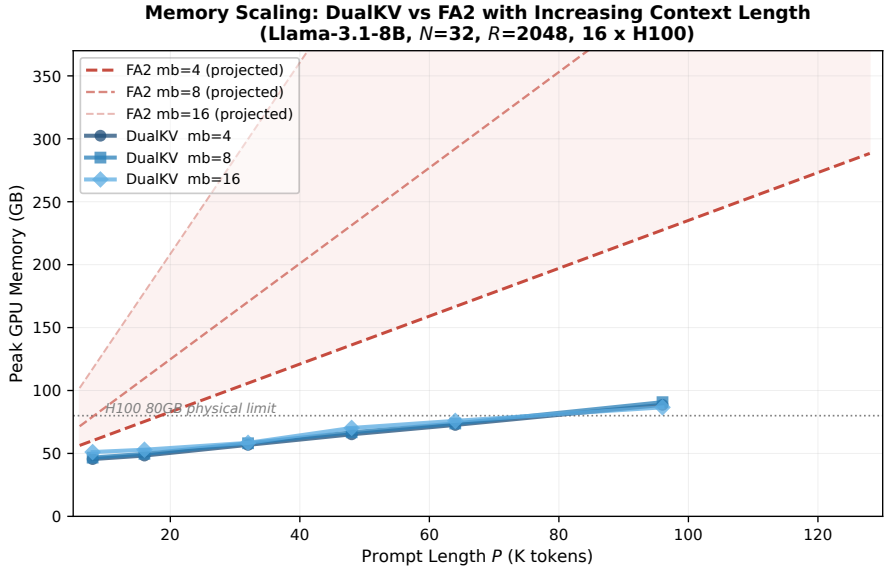

Figure 6: Memory scaling law: DualKV (measured, solid) vs. FA2 (projected, dashed). Llama-3.1-8B, N=32, R=2048, 16$\times$H100. DualKV's three micro-batch curves cluster tightly (mem $\propto P + \text{mb} \cdot R$), while FA2's diverge (mem $\propto \text{mb} \cdot (P + R)$). FA2 curves are projected from the fitted model since FA2 OOMs at these configurations.

E.4 Comparison with Prefix Grouper: Single-Layer Benchmark

To empirically validate DualKV's advantage over framework-level prefix sharing, we benchmark against Prefix Grouper [Liu et al., 2025] on a single Qwen3-8B decoder layer (forward + backward) on one H100-80GB GPU. Prefix Grouper decomposes attention into prefix self-attention and suffix cross-attention (2$\times$ FA2 calls), but operates on *padded* 2D tensors: all N sequences are padded to the longest response length and the full padded tensor passes through QKV projections, output projection, LayerNorm, and MLP. We give Prefix Grouper the best possible backend (padded `flash_attn_func`, not the default HuggingFace wrapper) and compare against both standard FA2 varlen (the baseline) and DualKV.

Setup. Qwen3-8B dimensions (H=32, H_k=8, d=128), bf16, response lengths $R \sim U(131, 2048)$. We sweep prompt lengths $P \in \{5059, 8192, 16384, 32768, 65536, 131072\}$ and micro-batch sizes $\text{mb} \in \{1, 2, 4, 8, 16, 32\}$. Timing: 2 warmup + 5 measured iterations. Correctness verified via `torch.allclose` against FA2 at smaller configs (both DualKV and PG pass; PG has zero error since it uses FA2 internally).

PG $\approx$ FA2 in latency. Prefix Grouper's attention optimization (avoiding redundant prefix-on-prefix $QK^\top$) is offset by the overhead of padding, KV expansion (`expand + contiguous`), and concatenation. Across all configs where both run, PG is within $\pm$20% of FA2 — sometimes slightly faster (attention savings dominate at large P), sometimes slightly slower (padding overhead dominates at large mb).

Table 12: Single-layer forward+backward: DualKV vs. Prefix Grouper (PG) vs. FA2. Time in ms, memory in GB. "DK/FA2" and "DK/PG" are DualKV's speedup ratios. OOM = out of memory on H100-80GB.

		Time (ms)			**DualKV Speedup**		**Peak Memory (GB)**	
P	mb	FA2	DualKV	PG	vs FA2	vs PG	DualKV	PG
5K	4	93.2	**36.6**	72.5	2.55×	1.98×	**4.0**	9.0
5K	8	153.3	**61.4**	156.9	2.50×	2.55×	**5.5**	18.1
5K	16	338.9	**110.4**	314.2	3.07×	2.85×	**9.0**	35.4
5K	32	597.2	**179.9**	626.3	3.32×	3.48×	**13.7**	69.8
8K	4	148.9	**57.2**	124.3	2.60×	2.17×	**4.9**	12.8
8K	8	245.4	**70.3**	232.1	3.49×	3.30×	**6.5**	25.8
8K	16	549.2	**128.2**	465.4	4.28×	3.63×	**9.9**	50.7
8K	32	OOM	**184.2**	OOM	∞	∞	**14.6**	—
16K	4	329.9	**108.9**	245.7	3.03×	2.26×	**7.5**	22.9
16K	8	553.0	**125.4**	434.9	4.41×	3.47×	**9.0**	45.9
16K	16	OOM	**205.8**	OOM	∞	∞	**12.5**	—
32K	2	408.0	**220.3**	308.1	1.85×	1.40×	**11.8**	22.1
32K	4	810.2	**246.4**	513.6	3.29×	2.08×	**12.5**	43.0
32K	8	OOM	**266.8**	OOM	∞	∞	**14.1**	—
65K	2	1173.5	**614.1**	773.3	1.91×	1.26×	**21.9**	42.3
65K	4	OOM	**653.9**	OOM	∞	∞	**22.7**	—
65K	16	OOM	**892.6**	OOM	∞	∞	**27.7**	—
131K	8	OOM	**2096.7**	OOM	∞	∞	**44.5**	—
131K	32	OOM	**2883.0**	OOM	∞	∞	**52.6**	—

DualKV is 2–4× faster than both. The speedup comes from processing fewer tokens through *every* operation in the layer. At the paper's primary config (P=8K, mb=8), DualKV processes 18K tokens vs. FA2's 75K and PG's 81K — a $4.2\times$ reduction that translates directly to a $3.5\times$ wall-clock speedup.

Memory gap is catastrophic for PG. PG allocates a padded $[\text{mb}, P + R_{\max}, H]$ tensor, consuming 3–5× the memory of DualKV's packed representation. At P=8K mb=16, PG uses 50.7 GB vs. DualKV's 9.9 GB ($5.1\times$ reduction); at mb=32, PG OOMs while DualKV fits in 14.6 GB.

Scalability. Of 54 total configurations tested (9 prompt lengths × 6 micro-batch sizes), DualKV runs 36 configs, while both FA2 and PG run only 21. Beyond P=131K, only DualKV can scale past mb=1 on a single H100.

Root cause: PG only optimizes attention. Attention accounts for $\sim 30\%$ of a decoder layer's FLOPs; the remaining 70% (QKV projections, output projection, MLP, norms) still operates on the full padded tensor with N duplicated prompt copies. DualKV eliminates redundancy across *all* operations by packing the prompt once in a varlen representation.

F Additional Experimental Figures

G DualKV Kernel Algorithms

This appendix gives full pseudocode for the DualKV forward and backward kernels (referenced from Sections 3.1–3.2). The listings elide K/V parallels: gradients, loads, and writes for V mirror those for K and are omitted for brevity.

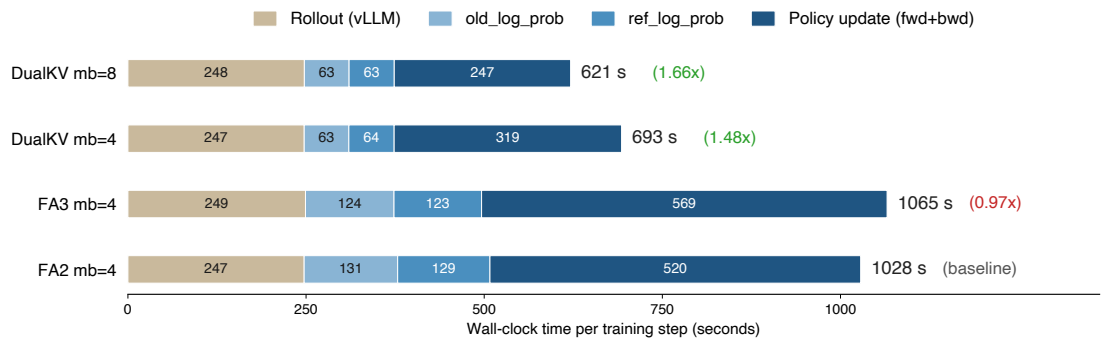

Figure 7: Per-phase breakdown of an average GRPO training step on LongReason (Section 4.2). DualKV compresses the three training-side phases (`old_log_prob`, `ref_log_prob`, `policy update`) by $\sim 2\times$ each; rollout latency is identical across all four configs. **(i)** the policy-update slice shrinks from 520 s (FA2, FA3) to 247 s ($2.1\times$ over FA2); **(ii)** the rollout slice is invariant ($\sim$247 s across all four configs); **(iii)** rollout's share of step time grows from 23% (FA2 mb=4) to 40% (DualKV mb=8) — DualKV shifts the bottleneck from the policy update to rollout generation.

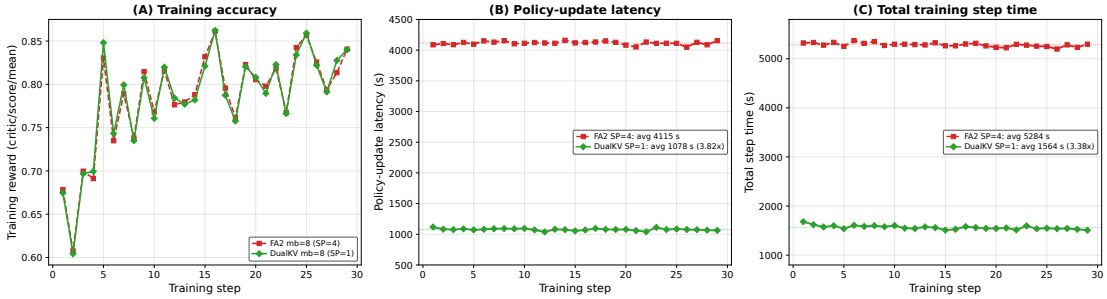

Figure 8: Multi-node MoE GRPO training on LongReason (Section 4.3; Qwen3-30B-A3B, 16×H100, 2 nodes), comparing DualKV mb=8 SP=1 against FA2 mb=8 SP=4 over the common 1–29-step window. **(A)** Per-step training reward — both configs track closely. **(B)** Per-step policy-update latency — DualKV averages 1078 s vs FA2's 4115 s ($3.82\times$). **(C)** Per-step total training step time — DualKV averages 1564 s vs FA2's 5284 s ($3.38\times$).

Algorithm 3 DualKV forward kernel (one thread block, assigned (m, b, h) = query-row block, batch, head)

Require: Q_m for batch b, head h; shared $K_c, V_c \in \mathbb{R}^{P \times H_k \times d}$; per-sequence $K_d^{(b)}, V_d^{(b)} \in \mathbb{R}^{R_b \times H_k \times d}$; tile size B_N; P (= `context_seqlen`)

$n_{\text{ctx}} \leftarrow \lceil P / B_N \rceil$
$n_{\text{dec}} \leftarrow \lceil R_b / B_N \rceil$
$n_{\max} \leftarrow n_{\text{ctx}} + n_{\text{dec}}$
Apply causal pruning: reduce $n_{\max}$ so no tile exceeds the causal horizon of Q_m
Init $m_i \leftarrow -\infty,\ \ell_i \leftarrow 0,\ \mathbf{o}_i \leftarrow 0$ ▷ online-softmax state per row in Q_m
for $n = n_{\max} - 1$ down to 0 **do** ▷ iterate K-tiles in reverse (FA2 convention)
  if $n < n_{\text{ctx}}$ **then** ▷ context tile: shared buffer
    $K_n, V_n \leftarrow K_c[nB_N : nB_N + B_N], V_c[\ldots]$
    $j_{\text{base}} \leftarrow n \cdot B_N$ ▷ logical K position
  else ▷ decoded tile: per-sequence buffer
    $K_n, V_n \leftarrow K_d^{(b)}[(n - n_{\text{ctx}})B_N : \ldots]$
    $j_{\text{base}} \leftarrow P + (n - n_{\text{ctx}}) \cdot B_N$
  end if
  Mask out-of-bounds lanes (partial tile at last context / last decoded tile)
  $S \leftarrow Q_m K_n^\top / \sqrt{d}$
  Apply causal mask using j_{base} (logical K position) vs. Q_m's logical row position
  $m_i^{\text{new}} \leftarrow \max(m_i,\ \text{rowmax}(S))$
  $P \leftarrow \exp(S - m_i^{\text{new}})$
  $\alpha \leftarrow e^{m_i - m_i^{\text{new}}}$ ▷ online-softmax rescale factor
  $\ell_i \leftarrow \alpha\, \ell_i + \text{rowsum}(P)$
  $\mathbf{o}_i \leftarrow \alpha\, \mathbf{o}_i + P \cdot V_n$
  $m_i \leftarrow m_i^{\text{new}}$
end for
$\mathbf{o}_i \leftarrow \mathbf{o}_i / \ell_i$ ▷ epilogue: normalize
Write $\mathbf{o}_i$ to $o[m]$
Save $L = m_i + \log \ell_i$ to `softmax_lse`[m] ▷ used by backward

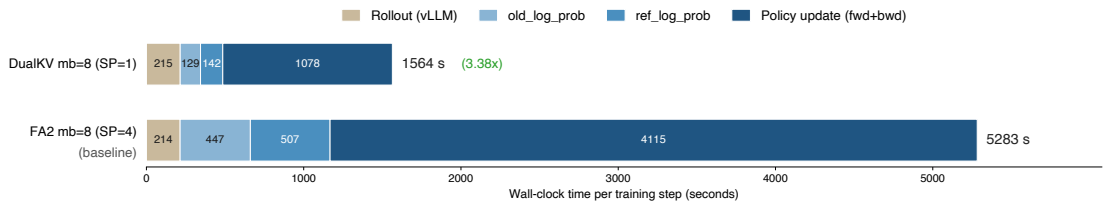

Figure 9: Per-phase breakdown of an average GRPO training step for the multi-node MoE experiment (Section 4.3). DualKV compresses the three training-side phases by 3.4–3.6× each; rollout latency is identical across both configs. The FA2 bar uses 4-way Ulysses SP — the smallest SP that avoids OOM at this scale — while DualKV runs at SP=1.

The backward main kernel (Algorithm 1) and the post-kernel context-gradient cast (Algorithm 2) are in Section 3.2.

H Kernel Interface: Python Signature and Tensor Shapes

This appendix gives the full Python signature of the DualKV kernel (referenced from Section 3) and the per-tensor shape contract.

```
out = flash_attn_dualkv_varlen_func(
    q, # (sum_Ri, H, d) decoded queries, packed varlen
    k_context, v_context, # (P, H_k, d) shared context KV (single copy)
    k_decoded, v_decoded, # (sum_Ri, H_k, d) per-sequence decoded KV, packed varlen
    cu_seqlens_q, # (N+1,) varlen offsets into q, k_decoded, v_decoded
    cu_seqlens_k_decoded, # (N+1,) same offsets (context is shared)
    max_seqlen_q, # int max R_i
    context_seqlen, # int = P causal offset: decoded query at index r
                    # has logical position P + r
    max_seqlen_k_decoded, # int max R_i
    softmax_scale=None, # float defaults to 1/sqrt(d)
    causal=True, # bool training always sets True
)
```

Listing 1: Signature of DualKV kernel interface `flash_attn_dualkv_varlen_func`

The key contract is that k_{ctx} and v_{ctx} (`k_context`, `v_context` in the listing) are stored once (not replicated N times), while q, k_{dec}, and v_{dec} are varlen-packed over the N decoded sequences. The kernel is implemented as a PyTorch `torch.autograd.Function`. Internally, the forward also computes a per-query softmax log-sum-exp tensor `softmax_lse` of shape $(H, \sum_i R_i)$ that is saved onto the autograd context for the backward pass (Section 3.2); it is not part of the user-facing return value.

I Implementation Details: Extending the Kernel to hd512 and SWA

This appendix documents the two kernel variants added for Gemma-4 support (Section 3): a head-dimension-512 full-attention path and kernel-native causal sliding-window attention (SWA) for head dimension 256. Both are compile-time specializations of the DualKV forward/backward kernels; the launcher dispatches on head dimension and window at runtime. FA2 supports only $d \leq 256$, so the d=512 path is what enables Gemma-4's global layers under shared-prompt deduplication.

Head dimension 512 (global layers). The forward launches `Flash_fwd_kernel_traits` with head dim 512, block sizes B_M=B_N=64, and 4 warps, restricted to full causal attention (no windowing). Because the d=512 tiles exceed the 48 KB static shared-memory limit, the kernel opts into dynamic shared memory via `cudaFuncAttributeMaxDynamicSharedMemorySize` at launch. The backward is the tighter case: at d=512 both shared memory and registers are under pressure, so it uses a smaller key block (B_M=64, B_N=32, 8 warps), holds V in registers, and single-buffers the KV pipeline (no double buffering) to fit within the device shared-memory-opt-in ceiling (queried at runtime via `cudaDevAttrMaxSharedMemoryPerBlockOptin`). These are the standard FA2 levers

for large head dims, applied within DualKV's two-region iteration. The d=512 path is used only for Gemma-4's full-attention (global) layers.

Sliding-window attention (local layers). Gemma-4's local layers use head dim 256 with a causal sliding window W. We add kernel-native SWA to the DualKV kernel (compile-time `Is_local`) for $d \leq 256$: within the two-region iteration, the window bound is applied in the logical position space (context positions $0 \ldots P-1$ followed by decoded positions $P+r$), so a decoded query at logical position $P+r$ attends only the last W logical keys spanning the prompt tail and its own response so far — identical semantics to running windowed FA2 over the contiguous $[K_c; K_d^{(i)}]$ sequence, but without replicating the shared context N times. Out-of-window context tiles are skipped at the block level. The result is verified mathematically equivalent to windowed FA2 (forward and backward) by the kernel unit tests.

Per-layer dispatch. A single DualKV forward over Gemma-4 dispatches per decoder layer: full-attention d=512 for the 10 global layers, windowed d=256 for the 50 sliding layers. The shared-prompt repack (Section 2.2) is layer-agnostic, so the same packed $P+NR$ sequence feeds both layer types.

J Comparison with Other Attention Variants

Readers familiar with attention-optimization literature may ask whether existing variants — paged attention, prefix caching, bifurcated attention, sequence-parallel methods, or sparse/linear attention — already address the shared-prompt redundancy we target. This appendix systematically positions DualKV against these variants along four capabilities relevant to RL policy-update training:

1. **Shared-prompt KV dedup**: the method stores the shared prompt's K, V once rather than N times.
2. **Shared-prompt compute dedup**: the method also processes the shared prompt through per-token operations (RMSNorm, QKV projection, RoPE, MLP, output projection) *once* rather than N times — i.e., eliminates the $(N-1) \cdot P$ tokens of compute outside attention.
3. **Training backward + autograd integration**: the method provides a differentiable backward pass, with gradients for all inputs compatible with PyTorch autograd (including gradient accumulation into the shared K_c, V_c across the N concurrent sequences).
4. **Mathematically exact**: the method produces outputs identical to standard attention up to floating-point rounding, not an approximation.

Table 13 summarizes the comparison. DualKV is the only variant that satisfies all four criteria simultaneously.

Table 13: Comparison of attention variants along four capabilities relevant to RL training. ✓ = satisfies; × = does not. DualKV is the only variant that is ✓ on all four.

Variant	KV dedup	Compute dedup	Training bwd	Exact
FlashAttention-2/3 [Dao, 2024, Shah et al., 2024]	×	×	✓	✓
Paged Attention [Kwon et al., 2023]	storage only	×	×	✓
SGLang RadixAttention [Zheng et al., 2024]	storage only	×	×	✓
Bifurcated Attention [Athiwaratkun et al., 2024]	attn. only	×	×	✓
Ring Attention [Liu et al., 2024a]	×	×	✓	✓
Ulysses SP [Jacobs et al., 2023]	×	×	✓	✓
Linear / sparse attention	×	×	✓	× (approx.)
KV-cache compression (H2O, SnapKV, etc.)	×	×	×	× (lossy)
DualKV (ours)	✓	✓	✓	✓

Inference-only variants (paged, RadixAttention, bifurcated). These methods share the shared-prompt KV at *storage* level (via block tables or memory-layout tricks) during inference. None defines a backward pass for training. Naively invoking a paged or bifurcated kernel in a training loop would fail because (i) PyTorch autograd cannot differentiate through block-table pointer indirection — the shared KV must be a proper tensor with a computation graph; (ii) with N concurrent sequences backward-propagating into the *same* K_c, V_c, the gradient accumulation requires race-free concurrent

writes with precision preservation (DualKV's fp32 atomic accumulation, Section 3.2); and (iii) they address attention only, leaving the $(N-1) \cdot P$ per-token cost in MLP, norms, and projections unaffected.

Sequence-parallel variants (Ring, Ulysses). These are orthogonal techniques for long-sequence training: they shard a *single* long sequence across multiple ranks via collective communication. They do not deduplicate the *shared prompt across multiple sequences* — that is a different axis of redundancy. Because the two axes are orthogonal, DualKV composes with sequence parallelism: its shared-prompt compression *delays* the onset of SP (fitting larger context and batch on fewer ranks) and eliminates SP entirely across a wide range of model-size $\times$ context regimes (Sections 4.2–4.3); for the extreme corner of a huge model at ultra-long context, where even the deduplicated sequence exceeds a single rank, the two are best used together. We implement and validate this composition and demonstrate it on Gemma-4-31B at 64K context (Section 4.5).

Approximate variants (linear, sparse, compressed). These trade accuracy for efficiency by replacing attention with $O(S)$ approximations or by pruning KV entries. They violate DualKV's "mathematically equivalent to standard attention" guarantee, and are orthogonal to shared-prompt dedup — they can be applied per-sequence independently of whether sequences share a prompt.

Summary. DualKV is the first attention variant to jointly provide (1) shared-prompt KV dedup, (2) shared-prompt compute dedup through the full transformer layer, (3) a mathematically exact training backward with gradient accumulation into the shared K_c, V_c, and (4) autograd-compatible integration with PyTorch. No existing technique can be dropped into a RL policy-update pipeline to replicate DualKV's savings.